THE UNIVERSITY OF DANANG

VN-UK INSTITUTE FOR RESEARCH AND EXECUTIVE EDUCATION





DEVELOPMENT OF A CARO WEB APPLICATION USING ADVERSARIAL SEARCH ALGORITHMS

BASIC PROJECT REPORT

Student: Nguyễn Tâm Đức

Student ID: 24020005

Supervisor: Nguyễn Văn Thọ

Class: CSE24

Academic year: 2026–2027

ABSTRACT

This project studies the design and implementation of a Caro web application together with a set of adversarial-search experiments and teaching materials. The report specifies a 15 × 15 Freestyle game, the board and move-history contracts, and the algorithms used to select and evaluate moves. It explains random and rule-based baselines, contiguous-run heuristics, tactical checks, Chebyshev candidate generation, soft candidate caps, Minimax and Alpha-Beta. It also treats iterative deepening, Zobrist hashing, transposition-table bounds, quiescence and incremental pattern counting in their corresponding reference or legacy implementations. Each algorithm is presented with its inputs, state, decision procedure, invariants, complexity, worked examples and limitations.

The application combines React interface components, a Pixi board, an authoritative controller and a Worker-based Rust/WebAssembly route. Request identity, round identity, cancellation, coordinate conversion and telemetry validation preserve the connection between a proposed AI move and the round in which it may be committed. The separate presentation website provides interactive explanations and Python/Pyodide examples. Its demonstrations are analysed individually rather than treated as measurements of the game engine.

Evaluation distinguishes published native experiments, binary/adapter parity, actual browser Worker observations, selected UI interactions and historical cross-language benchmarks. GitHub references identify immutable source and evidence snapshots. The analysis includes negative findings such as tactical moves discarded by a reference candidate cap, increased timing despite fewer searched nodes, and unsafe table reuse across a changed context. The resulting report connects algorithmic principles to concrete implementation while stating the boundaries of finite tests, source availability and reproducibility.

Keywords: Caro; Gomoku; Minimax; Alpha-Beta; candidate generation; Rust; WebAssembly; game-state consistency.

CONTENTS

CHAPTER 1. INTRODUCTION … 1

1.1 Background and motivation … 1

1.2 Objectives and research questions … 1

1.3 Scope, versions and source identity … 2

1.4 Method and report structure … 3

CHAPTER 2. ALGORITHMS AND DATA STRUCTURES … 4

2.1 Scope, evidence, and notation … 4

2.2 Board representation and legal updates … 5

2.3 Freestyle win detection … 8

2.4 Baselines: random play and rule-based play … 10

2.5 Static evaluation and terminal utility … 13

2.6 Candidate generation and a soft tactical cap … 16

2.7 Move ordering versus move selection … 19

2.8 Minimax search … 20

2.9 Alpha-beta pruning … 23

2.10 Iterative deepening and budgets … 26

2.11 Zobrist hashing and incremental state … 27

2.12 Transposition table: entries, bounds and context … 29

2.13 Quiescence search in the legacy TypeScript bot … 32

2.14 Legacy bitmask patterns and incremental line counts … 34

2.15 Difficulty configurations and the active request path … 36

2.16 Synthesis and limits of the algorithmic claims … 38

CHAPTER 3. SYSTEM ANALYSIS AND DESIGN … 40

3.1 Actors, use cases and functional requirements … 40

3.2 Non-functional requirements and acceptance boundaries … 43

3.3 Architecture and ownership … 43

3.4 Board state, history and legal transition contract … 45

3.5 Match phases and state transitions … 45

3.6 Asynchronous request protocol and cancellation … 47

3.7 Coordinate transformation and gesture interpretation … 48

3.8 Search-result contract and presentation of telemetry … 49

3.9 Verification scenarios derived from the design … 50

CHAPTER 4. IMPLEMENTATION … 52

4.1 Toolchain and project organization … 52

4.2 Accepted move processing … 52

4.3 AI dispatch, Worker execution and WASM hydration … 53

4.4 Board rendering and interface synchronization … 55

4.5 Presentation website and interactive explanation … 56

4.6 Python/Pyodide execution and four instructional programs … 56

4.7 Implementation traceability and limits … 59

CHAPTER 5. RESULTS AND EVALUATION … 60

5.1 Evaluation questions, evidence and boundaries … 60

5.2 Native board and search checks of the 23 October engine … 60

5.3 WebAssembly parity and browser observations … 62

5.3.1 Follow-up local application validation, 27 October … 64

5.4 Separate native reference experiments … 64

5.5 Historical cross-language and Week 4 datasets … 67

5.6 Presentation examples and evidence quality … 69

5.7 Threats to validity and interpretation … 69

CHAPTER 6. CONCLUSION AND FUTURE WORK … 71

6.1 Work completed and contribution … 71

6.2 Limitations … 71

6.3 Future development … 72

REFERENCES … 74

APPENDIX A. REPOSITORY INVENTORY AND EVIDENCE MAP … 82

APPENDIX B. EVIDENCE SUMMARY AND VERIFICATION REGISTER … 85

APPENDIX C. READING AND REPRODUCIBILITY PROCEDURE … 87

LIST OF FIGURES

Figure 2.1. Row-major board representation … 6

Figure 2.2. Search must restore its caller state … 8

Figure 2.3. Last-move scan along four axes … 10

Figure 2.4. Rule-based tactical priorities … 13

Figure 2.5. Contiguous-run weights of the archived Rust evaluator … 15

Figure 2.6. Radius and board edges affect candidate membership … 18

Figure 2.7. Alternative traversal orders on identical membership … 20

Figure 2.8. A two-level Minimax example … 23

Figure 2.9. Alpha-Beta cutoff on an unchanged tree … 25

Figure 2.10. Iterative deepening and incomplete work … 27

Figure 2.11. Incremental XOR and turn-key handling … 29

Figure 2.12. Bound semantics in a transposition table … 32

Figure 3.1. Responsibilities in the playable application … 44

Figure 3.2. Match phases and the separate AI waiting flag … 46

Figure 3.3. Inverse camera transform before board bounds checking … 48

Figure 4.1. Tactical and Rust routes converge at the guarded commit … 54

Figure 4.2. Union of clipped Chebyshev neighbourhoods in a five-stone example … 58

Figure 5.1. Recorded native engine nodes on the centre fixture … 62

Figure 5.2. Native reference node counts, with ranking cost excluded from bars … 66

Figure 5.3. Historical native-versus-Node benchmark on one midgame fixture … 68

LIST OF TABLES

Table 2.1. Difficulty configurations and recorded search behaviour … 36

Table 3.1. Functional requirements F01–F09 … 40

Table 3.2. Verification scenarios and expected outcomes … 50

Table 5.1. Native engine search on the centre fixture at depth two … 61

Table 5.2. Native reference search on the clustered fixture at depth two … 65

Table 5.3. Historical native Rust and TypeScript/Node benchmark … 68

Table A.1. Repository inventory and roles … 82

Table B.1. Evidence summary and verification status … 85

ABBREVIATIONS AND READING CONVENTIONS

AI — Artificial Intelligence.

DSA — Data structures and algorithms.

DFS — Depth-first search.

DOM — Document Object Model.

PRNG — Pseudorandom number generator.

PvP — Two people playing on the same application.

PvE — A person playing against the computer in this report.

TT — Transposition table.

UI — User interface.

WASM — WebAssembly.

Ply — One stone-placement move by one side.

Fixture — A deliberately prepared input position for a test or measurement.

Coordinates — Frontend (row,col) equals Rust (y,x); indices run from 0 to 14.

Evidence — Published source, a recorded experiment and an author observation are identified separately.

CHAPTER 1. INTRODUCTION

1.1 Background and motivation

Caro is a two-player placement game in which a small set of rules produces a large adversarial decision space. A player places one stone on an empty cell, attempts to complete a winning line and must anticipate the opponent's reply. The board representation is simple enough to implement explicitly, while the number of possible move sequences makes exhaustive search impractical beyond shallow depths. This combination makes the project suitable for studying data structures, recursive search, evaluation functions, pruning and the relationship between algorithmic results and an interactive application.

The project develops both a playable web application and material for explaining its algorithms. In the application, React supplies the surrounding interface, Pixi renders the board and a controller coordinates accepted moves, match state and AI requests. The AI path combines a TypeScript tactical check with a Worker-based WebAssembly engine. The repository also preserves a TypeScript search implementation, native Rust benchmark programs, baseline Rust engine code, weekly research reports and a separate presentation website. These modules form a cohesive architecture spanning interactive gameplay, WebAssembly search execution, and reproducible algorithmic benchmarks. [1] [2] [3]

The practical problem is to return a legal and useful move while maintaining coherent application state. A search that produces a high score is insufficient if it changes the caller's board, uses an incompatible win rule, blocks interaction unnecessarily or commits a result after the user has reset the round. The report therefore considers both the mathematical search problem and the application boundary at which a result becomes a move. It also distinguishes an observed counter from a configured limit and a native experiment from a browser measurement.

1.2 Objectives and research questions

The first objective is to specify the actual game rules and state transitions. A 15 × 15 board and a five-stone target do not uniquely define a Caro variant. The application's Freestyle rule accepts a contiguous line of at least five stones, including longer lines. Its implementation is explained from the last-move scan, with the assumptions under which that local test is valid. The second objective is to specify the AI algorithms precisely: their inputs, outputs, recurrence or decision

policy, candidate membership, state mutation and restoration, stopping conditions and computational cost. [4]

The third objective is to connect those algorithms to the implementation. The report identifies which component owns the board, who may commit a move, how history is replayed into the Rust/WASM object and how stale responses are rejected. The fourth objective is to evaluate the published experiment records without turning limited measurements into universal claims. In particular, it asks whether Minimax and Alpha-Beta agree on the same selected tree, whether ordering reduces total work or only visited nodes, and whether a cap can discard an immediate tactical completion.

These objectives lead to four research questions. RQ1 asks how legal board state and the Freestyle win condition can be represented and checked deterministically. RQ2 asks how evaluation, candidate generation and adversarial recursion produce a move, and which assumptions preserve correctness. RQ3 asks how pruning, ordering and caching affect the measured search, including counterexamples and context hazards. RQ4 asks how the native and browser components preserve a consistent game when requests are asynchronous. The evaluation chapter answers each question using the corresponding source implementation, mathematical invariants, and empirical benchmark records.

1.3 Scope, versions and source identity

The public project reference used throughout this English report is the immutable tducn110/basicproject revision 7e1e2d51321a4e6b2e26493c34f0a485d4de39cc. The application is under CaroGame; the report also covers present, the weekly folders, benchmark and research. The active game implementation under CaroGame is paired with the Rust board and engine modules in research/current_2026_10_23/baseline_engine/src/board.rs and the WebAssembly package in CaroGame/src/game/wasm/engine. [1] [5]

The repository integrates a TypeScript/React/Pixi frontend with a Rust engine compiled to WebAssembly, alongside native Rust benchmark harnesses and a legacy TypeScript search module. To maintain architectural clarity across these modules, this report explicitly distinguishes the active Rust/WASM application engine, the baseline Rust engine, the legacy TypeScript bot, and the native Rust reference benchmark library. [6] [7] [8]

The work excludes an online multiplayer server, a training pipeline, a learned neural policy and a demonstrated tournament ranking. These are not required to

explain the implemented algorithms. Browser and host behaviour are discussed only to the extent supported by source and recorded observations. A rule recommendation, a future experiment and an implemented feature are labelled separately. This scope allows the report to be detailed without claiming capabilities absent from the project.

1.4 Method and report structure

The analysis follows the path from an input to its visible consequence: pointer event, cell coordinate, legal move check, board mutation, win or draw evaluation, next-turn decision, AI request, validated response, committed move and updated display. Source inspection establishes the mechanism. Published test and measurement records establish what was exercised on their particular corpus and environment. Primary literature explains general search concepts; it is not used as proof that the project implements a feature. All project citations in the reference list resolve to GitHub resources rather than machine-specific file paths.

Chapter 2 specifies the data structures and algorithms. Chapter 3 gives the requirements, ownership boundaries and state/data flows. Chapter 4 explains the implementation of the playable application and presentation project. Chapter 5 evaluates the published native, WASM, Worker, UI and benchmark records with explicit limits. Chapter 6 states the achieved outcomes and development directions. The appendices inventory the repository and explain how to inspect or reproduce the work without silently substituting a different engine.

CHAPTER 2. ALGORITHMS AND DATA STRUCTURES

2.1 Scope, evidence, and notation

This chapter explains how a Caro position can be represented, tested for a result, evaluated and searched. It connects the concepts to evidence from the project while distinguishing three implementation contexts. The current application controller and frontend rules can be inspected directly in the application source. The repository's source excerpts and baseline modules document the core Rust engine functions, state representations, and search procedures used by the WebAssembly package [6]. Together with the compiled WebAssembly binary and TypeScript adapter contracts, these artifacts specify the active engine behavior and its boundary with the frontend controller [6], [7], [9].

The repository additionally contains a TypeScript bot implementation under the older bot module. It is called the legacy bot here because the current AIController constructs GomokuWasmEngine and sends the normal AI request through that adapter, with a separate tactical prepass for higher difficulties. Its quiescence search, iterative deepening, and transposition table provide concrete implementations of advanced game-tree optimizations analyzed in this chapter [10], [11]. A separate Rust reference engine has historical native research reports and test outputs. Results from that native reference library quantify algorithmic scaling and pruning behavior in controlled native experiments [8].

The notation throughout uses an N × N board with N = 15 and winning length K = 5 unless stated otherwise. Coordinates (x,y) mean column and row, respectively, with both in 0…14. In frontend code the equivalent coordinate is (row,col) = (y,x). A ply is one placed stone. Let B denote a board, p the side to move, opp(p) the other side, and m a legal move. The transition T(B,p,m) places p at m; it is defined only if the cell is empty and the game permits a move. Let A(B,p) be the moves the selected search policy actually generates, which may be a strict subset of the legal empty cells. A root player R is the player whose advantage is represented by a larger score. These distinctions matter because a proof about the generated search tree cannot establish a claim about every legal move if the generation policy has already excluded some moves.

The mathematical formulations and pseudocode specify each algorithm's inputs, state transitions, invariants, and complexity bounds, complemented by step-by-step worked examples and empirical measurements. Shannon's foundational model of

adversarial look-ahead and static horizon evaluation provides the theoretical framework for the depth-limited search architecture [12].

2.2 Board representation and legal updates

2.2.1 Matrix, index conversion, and state ownership

A position consists of 225 cells, each containing empty, X or O. In an array indexed as B[y][x], row-major conversion to a flat logical index is i = 15y + x; the inverse is x = i mod 15, y = floor(i/15). Hence the center (7,7) has index 112, and (14,14) has index 224. The system divides board representation across three specialized structures according to architectural role: (1) the Rust search engine (Board) stores a contiguous 15 × 15 grid of 225 one-byte repr(u8) cells (Empty = 0, Black = 1, White = 2) paired with move_count; (2) the match wrapper (Game) and frontend (GameState) store match progression and an ordered Move history; and (3) the legacy TypeScript bot (BotBoard) uses a 1D sentinel-padded array with directional bitmask segments. The rationale for this division is to decouple high-frequency recursive search from match-lifecycle management. Its primary benefit is O(1) direct indexing, O(1) in-place apply/undo during depth-first search, zero per-node heap allocation, and compact L1-cache locality; the trade-off is that the raw 225-cell matrix does not record chronological move order, requiring the outer Game/GameState layer to maintain history and full-board unindexed scans to take $O(N^2)$ time [13].

The archived Rust place excerpt validates bounds, rejects an empty stone, rejects an occupied cell, writes a cell and increments a move counter. The companion undo rejects an out-of-range or already-empty cell and otherwise clears it and decrements that counter [6]. Dividing responsibility between low-level cell mutation (Board::place and Board::undo) and match-level legality (Game::play_move and GameController) keeps inner-loop search operations lightweight while ensuring that turn alternation and post-win lockouts are strictly enforced on committed game moves.

A board and a game are different state owners. Cells and the count describe the position; turn, winner, and move history describe the match's progression. A history provides the information needed to restore actual game moves in reverse order. A recursive search temporarily changes a working board to explore alternatives, which must not silently become the player's actual move. The documented Rust search makes a copy at its outer entry, while its in-place search relies on restoration of the working board [6]. Cloning the root board once per search request isolates

exploratory tree mutations from the live game state, while in-place apply and undo avoid O(N²) board copying at internal nodes.

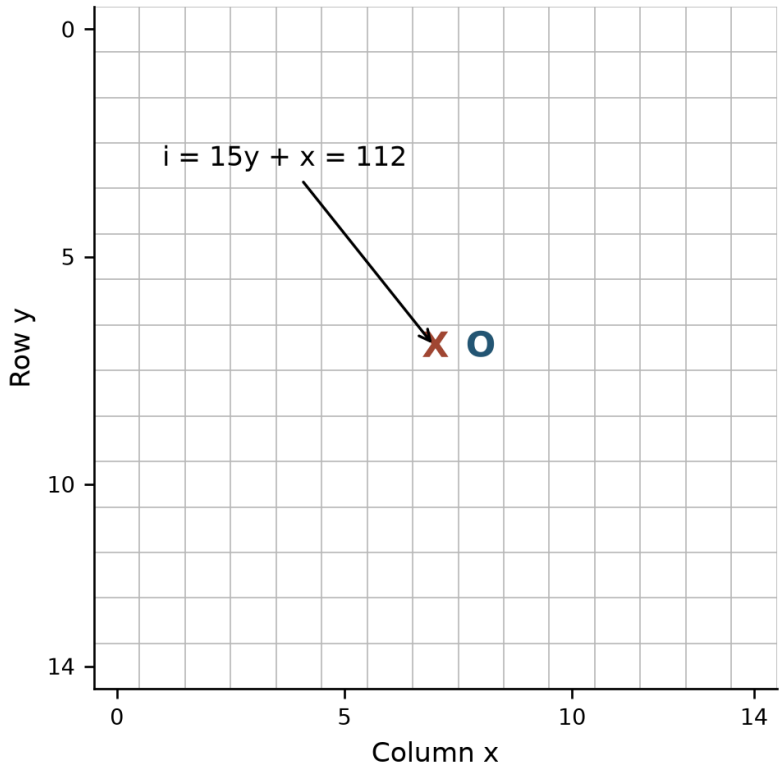


Figure 2.1. Row-major board representation

Source: CaroGame/src/game/core/GameState.ts [14].

2.2.2 Apply and undo

Inputs and output. APPLY(B,m,p) takes the working board, a coordinate and a nonempty player; it returns success or failure. UNDO(B,m) takes the coordinate of a previously applied move and returns whether removal was possible. The stronger search precondition is that undo is paired with the most recent successful hypothetical move on that branch. An individual undo that merely removes an occupied cell does not enforce the whole history discipline.

```
// Algorithm 2.1 (research/current_2026_10_23/baseline_engine/src/board.rs) — APPLY(B, (x,y), p):
    if x or y is outside the board, or p is empty: return failure
    if B[y][x] is not empty: return failure
    B[y][x] ← p
    B.move_count ← B.move_count + 1
    return success

UNDO(B, (x,y)):
    if (x,y) is outside the board or B[y][x] is empty: return failure
    B[y][x] ← empty
    B.move_count ← B.move_count - 1
    return success

VISIT_CHILD(B, p, m):
    require APPLY(B, m, p) succeeds
    try: value ← VISIT(B, opp(p), m)
    finally: require UNDO(B, m) succeeds
    return value
```

The finally in this explanatory version means every normal return, cutoff and supported abort must pass through restoration. In languages without such a construct, equivalent structured cleanup is necessary. Additional state such as a hash, side to move, cached pattern counts, last move or winner must also be restored by its actual owner. This paired apply/undo contract guarantees O(1) state transitions per branch while preserving parent-state integrity across arbitrary search depths.

Invariant and argument. Immediately before exploring each sibling, the working state must equal the state at the parent node. A successful apply changes exactly the selected empty cell and increments the count. Its matching undo clears that same cell and decrements the count; therefore these fields equal their former values. By induction over nested calls, assuming each recursive child restores every state field

it owns, the parent sees the same state for the next sibling. The induction depends on legal matching calls; it is not guaranteed if an exception bypasses undo or a different move is removed.

For example, let X occupy (7,7), O occupy (8,7), and the count be two. Trying X at (7,8) changes that cell and makes the count three. Undoing (7,8) returns both the cells and count to their initial values. Clearing the cell but leaving count three makes fullness checks unreliable; decrementing without clearing contaminates the next branch. With direct indexing, each operation is O(1) time and O(1) auxiliary space; copying an N × N board costs $O(N^2)$. Rejecting an occupied or out-of-bounds move must leave the board unchanged. Together with the inductive restoration invariant, automated root-restoration checks across 820 engine search runs confirm that the working board is returned unchanged after every search.

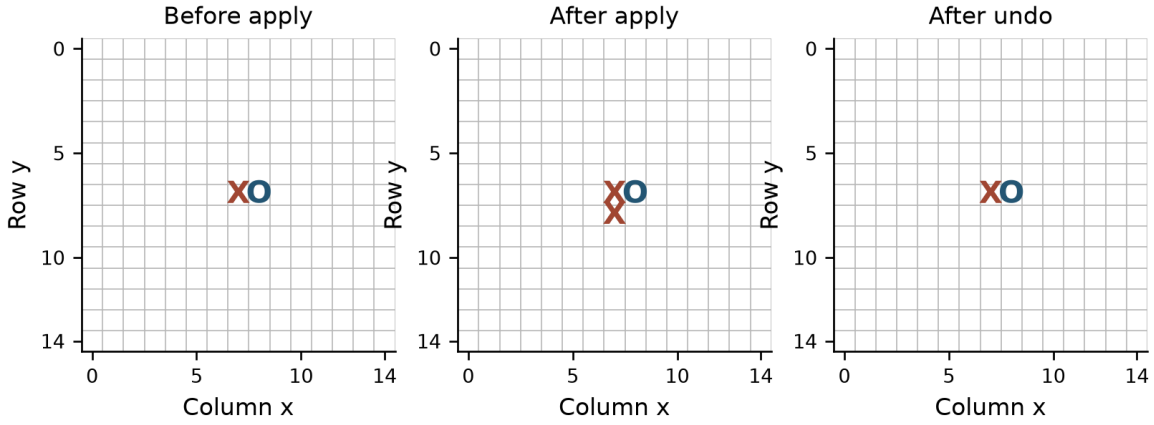


Figure 2.2. Search must restore its caller state

Source: research/current_2026_10_23/baseline_engine/src/board.rs [6].

2.3 Freestyle win detection

The frontend win rule treats a contiguous line of at least five stones as a win, including an overline of six or more. It checks four axes through the last move, with both directions on each axis. It does not require exactly five or impose an additional two-blocked-ends condition [4]. The term Caro is used for the application, but this particular rule is specified explicitly because games with similar names may use other winning conditions.

Inputs, outputs and preconditions. The procedure takes a board and the last placed coordinate m, and returns the winner and a collection of winning cells when a qualifying line is found, or no result otherwise. It assumes m refers to the move that may have just caused the transition. If the board was already won before that move,

testing only a line through m cannot detect every earlier win. A root-level validation of an arbitrary imported board needs a separate full-board check. The current frontend rule reads the player at m; an empty or invalid location produces no winner.

For directions D = {(1,0), (0,1), (1,1), (1,-1)}, let f_d and b_d count consecutive stones equal to the stone at m in the positive and negative direction. A win through the last move occurs exactly when some 1 + f_d + b_d ≥ K. To decide whether the threshold is reached, scanning at most K−1 matching cells in each direction is sufficient: any additional cells cannot change a negative result to a positive result after K−1 matching cells already exist on one side.

```
// Algorithm 2.2 (CaroGame/src/game/core/WinRules.ts) — WIN_AT(B, m, K):
    if m is out of bounds or B[m] is empty: return NONE
    p ← B[m]
    for d in [(1,0),(0,1),(1,1),(1,-1)]:
        cells ← [m]
        for sign in [+1,-1]:
            for step from 1 to K-1:
                q ← m + sign * step * d
                if q is outside or B[q] ≠ p: break
                append q to cells
        if length(cells) ≥ K: return (p, cells)
    return NONE
```

The scan invariant is that every collected cell is on the selected axis, adjacent to the preceding segment and occupied by p. Stopping at the first boundary or different cell prevents joining disconnected sequences. If a contiguous run through m has length at least K, either its collected positive and negative portions together reach K−1 neighbors or a single portion does, so the check succeeds. Conversely, K collected stones form a contiguous same-player segment through m, establishing the stated rule. The returned display collection may be truncated to K−1 stones per side; it is sufficient to prove a win but is not necessarily the full maximal overline.

Consider X X [X] X X on one row with the bracketed center as the last move. Each side contributes two; 1+2+2=5. For X X X [X] X X, the value exceeds five and

remains a win. An empty last cell returns no winner; a gap interrupts the run; edge and corner scans stop safely. Why last-move 4-axis ray scanning is used: because any newly formed winning line in an ongoing game must pass through the stone m just placed, scanning at most $K - 1 = 4$ steps in both directions (+1 and −1) along the 4 axes $D = \{(1,0), (0,1), (1,1), (1,-1)\}$ inspects at most $4 \times 2 \times 4 = 32$ neighboring cells. Benefits and trade-offs: this reduces terminal win detection at every recursive search node from an $O(N^2)$ full-board scan ($225 \times 4 = 900$ directional checks) to $O(K) = O(1)$ time and $O(K)$ space, yielding a major speedup inside Minimax and Alpha-Beta; the trade-off is that it relies on the inductive invariant that no prior move had already won, so arbitrary imported positions require a one-time $O(N^2)$ root validation.

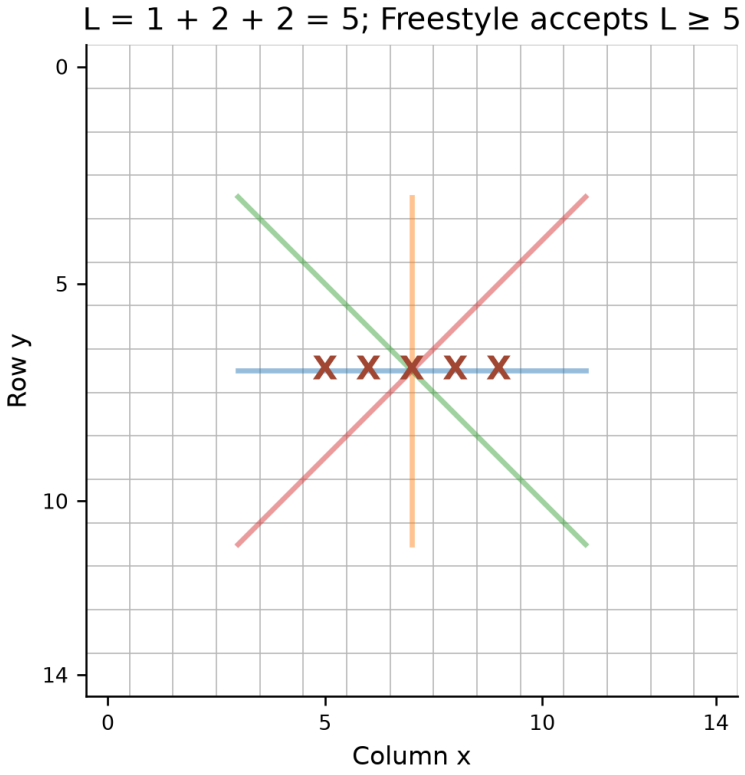


Figure 2.3. Last-move scan along four axes

Source: CaroGame/src/game/core/WinRules.ts [4].

2.4 Baselines: random play and rule-based play

2.4.1 Random baseline

A random baseline makes the meaning of added search explicit: it selects without comparing a tree of future replies. The documented Rust baseline first checks for a terminal position or an invalid empty player; for Easy it gathers legal empty cells,

uses a seeded pseudo-random value modulo the number of cells, and evaluates the resulting move for the returned result. That evaluation occurs after selection and does not choose the Easy move [6].

```
// Algorithm 2.3 (research/current_2026_10_23/SOURCE_EXCERPTS.md) —
RANDOM_MOVE(B, p, seed):

    if p is empty or B is terminal: return NONE

    L ← all legal empty cells in B

    if L is empty: return NONE

    i ← SEEDED_INTEGER(seed) mod length(L)

    return L[i]
```

The input includes the seed because pseudo-random generation is deterministic for a particular generator state and input. The output is either an element of L or no move. With a board containing only one stone, |L|=224; a one-element list always returns its sole entry. A full or terminal board must not perform modulo zero. The legality argument is immediate from construction of L, provided the game-level preconditions and list are correct. Using a deterministic 64-bit splitmix-style mixer over the legal move list ensures fast O(1) index selection after $O(N^2)$ empty-cell enumeration.

Gathering all cells takes $O(N^2)$ time and $O(N^2)$ space in the worst case; selection after that is O(1). A post-selection evaluation has its own cost. A fixed seed makes the selection reproducible under the same generator and ordering, not necessarily across different implementations or changed inputs. A random policy does not promise to notice an immediate win or threat.

2.4.2 Immediate tactics and one-ply heuristic

The documented Normal baseline first obtains candidates, then chooses an immediate win when available, otherwise an immediate blocking coordinate, and otherwise compares the evaluations of candidate moves. These are priorities, not a multi-ply adversarial proof. The candidate assessment is the source of the immediate-win and block markers [6]. The current frontend also has a separate tactical resolver for Hard, Very Hard, Expert and Master before it requests the WASM engine. Easy and Normal skip that frontend prepass. A tactical response reports its own source and zero Rust search work; it must not be presented as a completed alpha-beta search [10].

```
// Algorithm 2.4 (research/current_2026_10_23/SOURCE_EXCERPTS.md) —
RULE_BASED(B, p, policy):
    if B is terminal or p is invalid: return NONE
    C ← SELECT_CANDIDATES(B, p, policy)
    if an m in C wins immediately for p: return m
    if an m in C is an immediate winning cell for opp(p): return m
    best_move ← NONE; best_score ← -infinity
    for m in C:
        APPLY(B,m,p)
        try:
            score ← TERMINAL_OR_EVALUATE(B,p,m)
        finally:
            UNDO(B,m)
        if best_move is NONE or score > best_score:
            best_move ← m; best_score ← score
    return best_move, best_score
```

The decision invariant after processing the first j non-tactical candidates is that best_score is the greatest observed value among those j moves and the board is restored. A separate invariant places the win and block checks before the one-ply loop. Thus the routine picks the best score among the evaluated candidates when neither tactical condition applies. Correctness of the tactical marker itself depends on the assessment recognizing the corresponding winning geometry and on the move being included in C.

For a concrete threat, suppose X occupies row 0, columns 0–3, column 4 is empty, and O is to move. The left endpoint lies outside the board; placing O at (4,0) blocks the only immediate completion along this row. If the same four consecutive X stones are in the interior with both endpoints empty, there are two distinct winning cells. A single O move can occupy one cell, not both. Retaining both choices avoids losing information but does not establish that either saves the game. Similarly, an immediate winning move for O takes priority over blocking if both are available under the algorithm's ordering.

If k candidates are generated, one-ply evaluation costs roughly O(k × evaluation-cost), in addition to candidate generation and ordering. A constant-time

local win scan does not make the whole rule-based procedure constant time. A nonterminal board with no generated move requires a defined fallback or no-move response; terminal handling must precede any selection. A short tactical prepass is not a substitute for searching through arbitrary combinations.

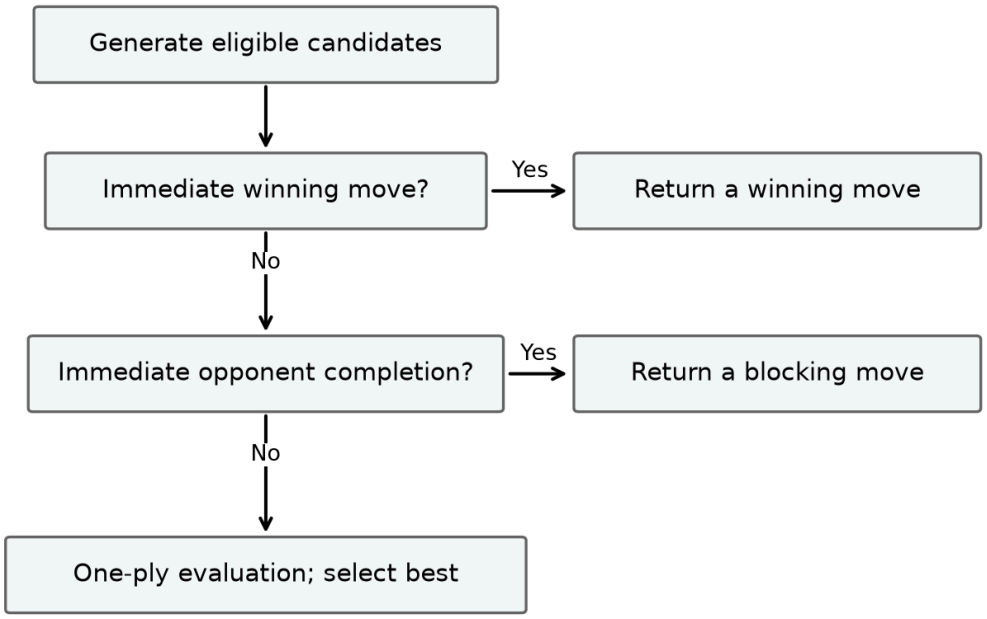


Figure 2.4. Rule-based tactical priorities

Source: research/current_2026_10_23/baseline_engine/src/ai.rs [7].

2.5 Static evaluation and terminal utility

2.5.1 Evaluation of nonterminal positions

A search with finite depth can stop before a winner exists. Its evaluator assigns an approximate score to that horizon position. In the documented modified Rust snapshot, evaluate scans maximal contiguous runs in four directions, adds the root player's run weights, subtracts the opponent's, and adds or subtracts a center preference. The snapshot counts a run once per axis by skipping a cell when the previous cell on the same axis has the same color. It then clamps the final heuristic to a range below terminal scores [6].

Let R_p(B) be maximal runs of p over the four axes, len(r) their length and open(r) their number of empty endpoints. Let c(x,y)=max(0,14−|x−7|−|y−7|) and C(B,p) be p's sum of c minus the opponent's. The descriptive formula is

$$E_{raw}(B,p) = \Sigma[r \in R_p(B)]\ w(len(r),open(r)) - \Sigma[r \in R_{opp}(B)]\ w(len(r),open(r)) + C(B,p).$$

The publicly archived Rust baseline defines the following weights for two open ends and one open end, respectively: length at least five, 1,000,000 and 1,000,000; length four, 50,000 and 5,000; length three, 5,000 and 500; and length two, 200 and 50. A run of five or more has the same weight even when both ends are closed; otherwise unmatched cases receive zero from the run-weight function [15]. Why exponential pattern weights are used: scaling weights by an order of magnitude per stone—from 50/200 for two-stone runs to 500/5,000 for three-stone runs, 5,000/50,000 for four-stone runs, and 1,000,000 for five-or-more—ensures that an unstoppable open four (50,000) or double-three threat decisively outweighs multiple scattered two-stone formations, while the Manhattan center term $c(x,y) = \max(0, 14 - |x - 7| - |y - 7|)$ breaks positional ties in the opening.

```
// Algorithm 2.5 (research/current_2026_10_23/baseline_engine/src/eval.rs) — EVALUATE(B, p):
    score ← 0
    for each occupied cell s:
        for d in the four axes:
            if preceding cell along d has the same color: continue
            (length, open_ends) ← scan maximal same-color run
            value ← WEIGHT(length, open_ends)
            if B[s] is p: score ← score + value
            else: score ← score - value
        score ← score + center contribution signed by color
    return CLAMP(score, -H, H)
```

The loop invariant is that score equals the signed contribution of all starts already visited, plus center terms for occupied cells already visited. Every maximal run has exactly one start in each orientation, so the preceding-cell test avoids counting that run repeatedly. Because every maximal contiguous run has a unique starting cell along each axis, the preceding-cell guard (skipping cell s when $s - d$ holds the same color) guarantees that each run is weighted exactly once per axis without double-counting sub-segments.

For an illustrative board with X alone at (7,7), its singleton has no run weight and its center term is 14, so E(B,X)=14. Add X at (8,7) while leaving the remainder empty: the horizontal length-two run has two open ends and contributes 200; center terms are 14 and 13, so the value is 227. A blocked endpoint reduces open(r) from 2 to 1 (or 0 when blocked on both sides), shifting the run weight down by an order of magnitude; board boundaries act as closed endpoints.

Scanning all maximal runs across a constant number of directions is $O(N^2)$: although a run is traversed, each occupied cell participates in at most one maximal run per direction. Auxiliary space beyond the board is O(1) in the described scan. The evaluator recognizes contiguous runs; a gap such as XX_XX is not automatically a single contiguous run, even though a separate move test can recognize the winning gap. Benefits and trade-offs of maximal contiguous-run scanning: it evaluates any board in $O(N^2)$ time and O(1) auxiliary memory without dynamic allocation; the trade-off is that single-gap threats (such as XX_XX) are not scored as a single contiguous run at depth 0, relying instead on multi-ply look-ahead recursion and the candidate generator's tactical assessment to resolve split completions.

Illustrative run	Archived Rust weight
.XXXXX.	1,000,000
.XXXX.	50,000
0XXXX.	5,000
.XXX.	5,000
0XXX.	500
.XX.	200
0XX.	50

Figure 2.5. Contiguous-run weights of the archived Rust evaluator

Source: research/current_2026_10_23/baseline_engine/src/eval.rs [7].

2.5.2 Terminal states and the score hierarchy

Terminal outcomes must be evaluated before a depth cutoff. In the engine implementation, the terminal base constant is M = 100,000,000, the ply distance from the root is q ≤ 225, and nonterminal heuristic scores are clamped to (−H, +H) with H = M − 225 − 1 = 99,999,774 [6]. The result from root perspective is M−q for a root win, −M+q for a root loss, and zero for a draw. An unfinished board has no terminal utility and is evaluated only when the search horizon is reached [6].

```
// Algorithm 2.6 (research/current_2026_10_23/SOURCE_EXCERPTS.md) —
TERMINAL_UTILITY(winner, full, root, ply):

    q ← min(ply, 225)

    if winner = root: return M - q

    if winner is the opponent: return -M + q

    if full: return 0

    return NONE
```

The clamp ensures every nonterminal heuristic lies between −H and H. Since M−225 = 99,999,775 > H, even the slowest represented win exceeds a nonterminal score; similarly a loss is below every nonterminal score. A win at q=2 receives 99,999,998 and a win at q=4 receives 99,999,996, so maximizing prefers the earlier win. Conversely, if a loss is unavoidable, increasing q makes its negative score less severe and thus delays the loss. This preference follows from the formula; it is conditional on the search actually reaching and correctly identifying those terminal outcomes. The winner must be decided before a full-board draw when the last empty cell completes a line.

2.6 Candidate generation and a soft tactical cap

2.6.1 Spatial neighborhood

Considering every empty cell creates a large branching factor. A candidate policy can instead choose empty cells close to existing stones. For cells a and b, Chebyshev distance is $d_\infty(a,b)=\max(|a_x-b_x|, |a_y-b_y|)$. With radius r, a spatial candidate is an empty cell within distance r of at least one occupied cell. The empty board has a special center choice, and a full board has no move. The exact selection, fallback and ranking below describe the archived modified-engine policy, not the legacy bot's different forcing-move generator [6].

```
// Algorithm 2.7 (research/current_2026_10_23/SOURCE_EXCERPTS.md) —
RAW_CANDIDATES(B, r):
    if B is full: return []
    if B has no stones: return [center]
    seen ← N × N false array; C ← []
    for each occupied cell s in B:
        for each q in the radius-r square around s clipped to the
board:
            if B[q] is empty and not seen[q]:
                seen[q] ← true
                append q to C
    return C
```

For a stone at (7,7), radius one gives a 3 × 3 square with eight empty candidates; radius two gives a 5 × 5 square with 24. With the single stone at (0,0), the radius-two square is clipped to a 3 × 3 corner and yields eight candidates. Multiple stones may nominate the same cell, but seen prevents duplicates. If a radius produces no raw candidates on a nonfull board, the documented selection code falls back to legal moves, avoiding an occupied placeholder. An empty player and a full board return no candidates [6].

The membership invariant says that each appended coordinate is on the board, empty and not previously appended. Conversely, every empty cell in a visited neighborhood is appended on its first encounter. This proves correctness for the defined neighborhood, not equivalence to all legal moves. With m occupied cells, scanning the board and their neighborhoods costs $O(N^2 + m(2r+1)^2)$ before assessment; the seen matrix and candidate list use $O(N^2)$ space. The domain can be clipped at the board edge. Benefits and trade-offs of Chebyshev spatial pruning: restricting moves to $d_\infty(a,b) = \max(|a_x - b_x|, |a_y - b_y|) \leq r$ reduces the branching factor from ~220 empty cells in the opening to 8–36 local candidates (a 72.39%–83.64% reduction), making multi-ply search tractable; the trade-off is that distant non-local sacrifices outside radius r are excluded from the search tree.

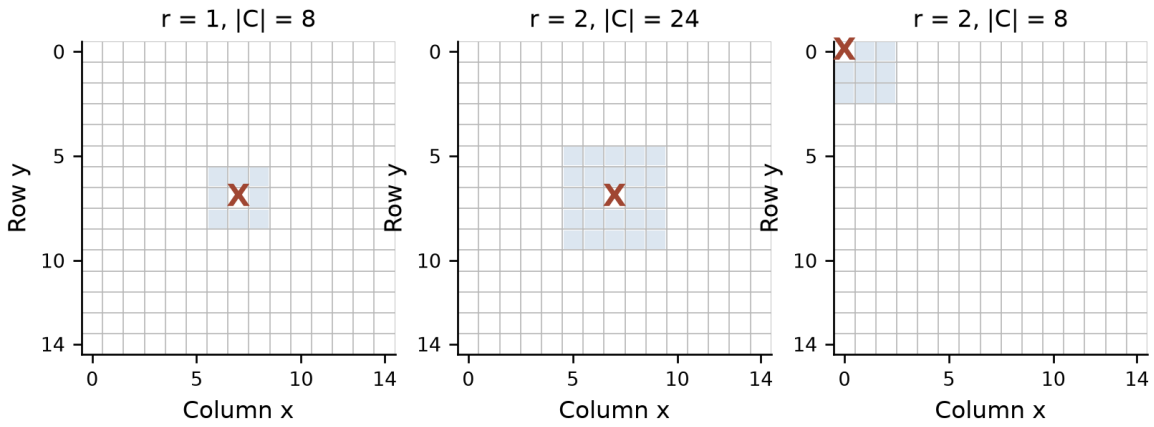


Figure 2.6. Radius and board edges affect candidate membership

Source: research/current_2026_10_23/SOURCE_EXCERPTS.md [6].

2.6.2 Tactical preservation and membership

For each raw move, the documented select_candidates assesses whether it wins for the moving side, blocks an immediate opponent win, and assigns a quick ranking score. Sorting prioritizes a win, then a block, then descending score, with row-major coordinates used to break ties. If a cap is configured, the ranked list is truncated at max(cap,1,tactical_count), so every assessed immediate winning or blocking coordinate remains even if the number of such coordinates exceeds the cap. After selecting membership, disabling move ordering sorts the retained set by coordinate rather than changing its members [6].

```
// Algorithm 2.8 (research/current_2026_10_23/SOURCE_EXCERPTS.md) —
SELECT_CANDIDATES(B, p, policy):
    if p is empty or B is full: return empty set
    raw ← RAW_CANDIDATES(B, policy.radius)
    if raw is empty and B is not full: raw ← LEGAL_MOVES(B)
    ranked ← ASSESS each move for win, block, and quick score
    sort ranked by win first, block second, score descending,
coordinate
    tactical_count ← count of entries marked win OR block
    if policy.cap exists:
        retain first max(policy.cap, 1, tactical_count) entries
    if ordering is disabled: sort retained entries by coordinate
    return retained entries and assessment metadata
```

Let C be the raw set, T its marked tactical subset, and c the configured cap normalized to at least one. The selected set is the union of T and enough highest-ranked non-tactical members to reach c when possible. If |T|=2 and c=1, both tactical moves remain, so this is a soft cap, not an absolute maximum. The proof depends on tactical entries being ordered before every non-tactical entry and on every relevant move first being in raw. It does not show that all useful multi-ply tactical moves are preserved, or that a move excluded by the spatial policy is later recovered by the cap.

For k raw candidates, assessment takes O(kK) when the relevant directional scan is bounded by K, sorting O(k log k), and storage O(k), in addition to spatial generation. With fixed K=5 the scan term is linear in k. Candidate assessment and full-board evaluation are different operations; a counter named ordering evaluations should be interpreted using its own recorded definition. Why a soft tactical cap C' = T ∪ Top_max(0, c − |T|)(C \ T) is used instead of a hard cap: a hard truncation at cap c can discard critical winning or blocking moves if their static heuristic rank falls below index c—as demonstrated in the cap_loss.json benchmark where a hard cap of 6 discarded both winning endpoints (7,2) and (7,7) out of 20 candidates. By taking max(cap, 1, tactical_count), the soft cap guarantees 100% retention of immediate 1-ply wins and blocks (T) while bounding non-tactical branching to at most c candidates [16].

2.7 Move ordering versus move selection

A search traverses the retained moves in an order. Ordering permutes an already chosen set; filtering or capping changes its membership and therefore the tree itself. In a complete minimax evaluation of the same deterministic tree, any permutation produces the same value, although a tied best move can differ if the first best move is retained. In alpha-beta search, ordering influences how soon useful bounds are discovered and can alter the number of cutoffs, while preserving the root value under the algorithm's assumptions [17].

```
// Algorithm 2.9 (research/current_2026_10_23/SOURCE_EXCERPTS.md) —
ORDER(retained_moves, rank):

    calculate a rank key for each retained move

    sort by the chosen key with a deterministic tie breaker

    return the same moves in the new order
```

For example, if A, B and C have quick ranking scores 10, 50 and 30, score order is B, C, A, while coordinate order may be A, B, C. Neither order establishes that B is

ultimately the best move. If all are still visited under minimax, only traversal changes. Alpha-beta may finish earlier after B supplies a strong bound. Sorting k moves costs O(k log k), plus the cost of computing keys and O(k) storage in a conventional array implementation. Benefits and trade-offs of move ordering: examining immediate wins, blocks, and high-adjacency moves first maximizes early $\alpha \geq \beta$ cutoffs and reduces visited nodes by 44.68%–80.22% in depth-2 benchmarks; the trade-off is the O(k log k) sorting and key-assessment overhead at each node, which must remain lightweight so ranking cost does not exceed pruning savings.

The documented candidate selection uses fast local information for ranking; the historical reference experiment uses its own evaluation and ordering behavior [6], [8]. Comparisons should hold the board, depth, scoring, candidate membership and terminal rules fixed, changing only ordering. Equal values but different tied moves can both be consistent. A comparison that changes both cap and ordering cannot isolate ordering's effect.

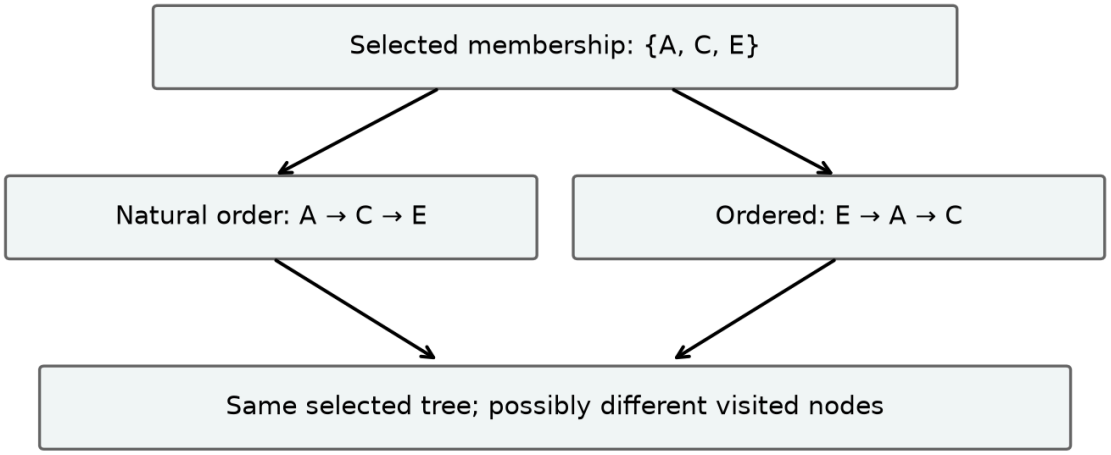


Figure 2.7. Alternative traversal orders on identical membership

Source: research/current_2026_10_23/SOURCE_EXCERPTS.md [6].

2.8 Minimax search

2.8.1 Recurrence and contract

Minimax assumes two alternating decision-makers. MAX chooses a move to maximize a score from the root player's perspective; MIN chooses to minimize it. Its input is the current board, the side to move, a nonnegative depth d in plies, the last move when available, a candidate policy and an evaluator. The output is a value

and, at the root, a selected move, or no move for an already finished game. State includes the mutable working board, remaining depth, root perspective and recursion stack. Search must begin from a board compatible with the chosen rules, or validate it separately.

Define U(s,R) for terminal states, E(s,R) for a nonterminal horizon, and transition T(s,a). With a fixed deterministic candidate function A, the depth-limited value is

V(s,d) = U(s,R) if s is terminal;

V(s,0) = E(s,R) if s is not terminal;

V(s,d) = max[a∈A(s)] V(T(s,a),d−1) if MAX moves;

V(s,d) = min[a∈A(s)] V(T(s,a),d−1) if MIN moves.

An empty generated set for a nonterminal board must have a specified behavior. The documented value routine returns zero for an empty list, while candidate fallback is intended to avoid this on a nonfull valid position. That fallback convention should not be silently reinterpreted as a win or as a comprehensive validation of arbitrary positions [6].

```
// Algorithm 2.10 (research/current_2026_10_23/SOURCE_EXCERPTS.md)
— MINIMAX(B, player, root, depth, last, ply):
    t ← TERMINAL_UTILITY(WIN_AT(B,last), IS_FULL(B), root, ply)
    if t exists: return t
    if depth = 0: return EVALUATE(B,root)
    moves ← SELECT_CANDIDATES(B,player)
    if moves is empty: return 0   // documented convention for this
case
    best ← -infinity if player = root else +infinity
    for m in moves:
        require APPLY(B,m,player)
        try: v ← MINIMAX(B,opp(player),root,depth-1,m,ply+1)
        finally: require UNDO(B,m)
        if player = root: best ← max(best,v)
        else: best ← min(best,v)
    return best
```

At the root, test each allowed root move, remember an action that achieves the greatest value, and restore the board after every branch. Initial root validation may need to scan for a pre-existing winner, whereas recursive checks can use the just-played move under the earlier precondition. The engine implements a unified recursive search kernel configurable as either full-tree Minimax or Alpha-Beta pruning over the same candidate generator and evaluator [6].

2.8.2 Proof, example and complexity

Use induction on d. For terminal nodes or d=0 the procedure returns exactly the corresponding definition. Assume recursive results equal V for depth d−1 and that each apply/undo restores the parent. At a MAX node the loop's best value after j children is the maximum of the first j child values; at completion it is the maximum over A. The MIN case is symmetric. Each recursive call reduces d, so a finite candidate list guarantees termination. This proves evaluation of the specified finite tree if transition, terminal detection, evaluation and candidate generation satisfy their contracts; it does not prove the evaluator predicts the outcome of an unrestricted game.

In an abstract depth-two tree, MAX has actions A and B; MIN's leaves under A have values 3 and 5, and under B have values 2 and 9. MIN returns 3 for A and 2 for B; MAX chooses A with 3. This two-ply trace illustrates how alternating MAX and MIN backups propagate exact horizon utility values to the root decision.

If the maximum number of generated moves is b at each level, the full tree has O(b^d) nodes for b>1; actual branching may vary by state, cap and terminal events. Depth-first exploration uses O(d) stack frames plus the live candidate lists, conservatively O(db) move entries and a working board of $O(N^2)$, not storage for the whole tree. Each node also incurs candidate-generation or evaluation costs, which can be substantial. If the terminal is reached sooner, recursion stops early. Benefits and trade-offs of Minimax: it provides an exact game-theoretic baseline over the generated finite tree with $O(db + N^2)$ memory, but its exponential O(b^d) time complexity restricts unpruned search to shallow depths.

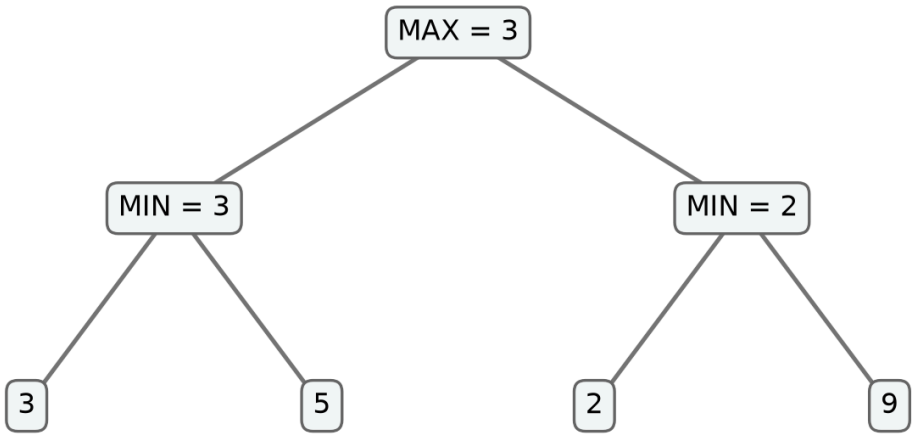


Figure 2.8. A two-level Minimax example

Source: Depth-limited Minimax formulation [12].

2.9 Alpha-beta pruning

2.9.1 Bounds and pseudocode

Alpha-beta pruning computes the same root value as minimax on the same finite deterministic tree under a consistent utility and candidate policy, while sometimes skipping subtrees. alpha is the best lower bound MAX can already secure in the current ancestor context; beta is the best upper bound MIN can already secure. At a MAX node, update alpha after observing a child; at a MIN node, update beta. When alpha ≥ beta, the remaining siblings cannot improve the relevant ancestor's decision. Knuth and Moore analyze alpha-beta and the effects of move ordering under stated assumptions [17].

```
// Algorithm 2.11
(research/current_2026_10_23/baseline_engine/src/ai.rs) —
ALPHABETA(B, player, root, depth, last, ply, alpha, beta):
    t ← TERMINAL_UTILITY(WIN_AT(B,last), IS_FULL(B), root, ply)
    if t exists: return t
    if depth = 0: return EVALUATE(B,root)
    moves ← SELECT_CANDIDATES(B,player)
    if moves is empty: return 0
    best ← -infinity if player = root else +infinity
    for m in moves:
        require APPLY(B,m,player)
        try: v ←
ALPHABETA(B,opp(player),root,depth-1,m,ply+1,alpha,beta)
        finally: require UNDO(B,m)
        if player = root:
            best ← max(best,v); alpha ← max(alpha,best)
        else:
            best ← min(best,v); beta ← min(beta,best)
        if alpha ≥ beta: break
    return best
```

Here restoration occurs before a cutoff is tested. The caller starts with an unbounded window when it needs a root result. A value returned from an internally cut-off node may represent a bound rather than an exact independent value. This distinction becomes essential when such results are cached. The code excerpt increments a cutoff counter at a break; that event count is not the total number of theoretical descendant nodes omitted [6].

2.9.2 Worked cutoff and correctness boundary

Use the same abstract tree. Search A first: its MIN node sees 3 and 5, returns 3, and MAX updates alpha to 3. At B, MIN first sees 2, so beta becomes 2. Because beta ≤ alpha, B cannot supply MAX a value exceeding the already available 3: MIN can choose the 2 branches. The remaining B child, whose illustration value is 9, is unnecessary for choosing the root action. The root answer remains A, 3. Visiting the

children in another order may produce fewer or no cutoffs without changing the full-tree result.

The invariant is that alpha and beta summarize bounds supplied by already considered alternatives and ancestors. Once they cross, further siblings at the current node cannot alter the ancestor choice under the minimax ordering. Combined with the minimax induction and the restoration invariant, pruning preserves the root value on the same tree. A practical check compares minimax and alpha-beta with identical positions, turns, depth, terminal rule, evaluator, candidate set and preferably deterministic tie behavior. If the set of candidate moves differs, the comparison tests two different games rather than pruning alone. Across all 408 paired native engine comparisons and 112 reference comparisons, Alpha-Beta and Minimax return identical root scores while Alpha-Beta visits 35.62%–77.60% fewer nodes.

Worst-case O(b^d) remains possible, for instance with unhelpful ordering. Idealized ordering in a uniform tree can reduce the effective leaf growth toward O(b^(d/2)); this is a model, not a promise of a particular elapsed time. The recursion stack and live candidate lists have the same conservative O(db) bound as depth-first minimax, in addition to the board. Cutoff at equality is valid for the value, though tie selection and visited-node counts can depend on ordering. The algorithm alone does not address deadline, collision safety, illegal input or selective candidate loss.

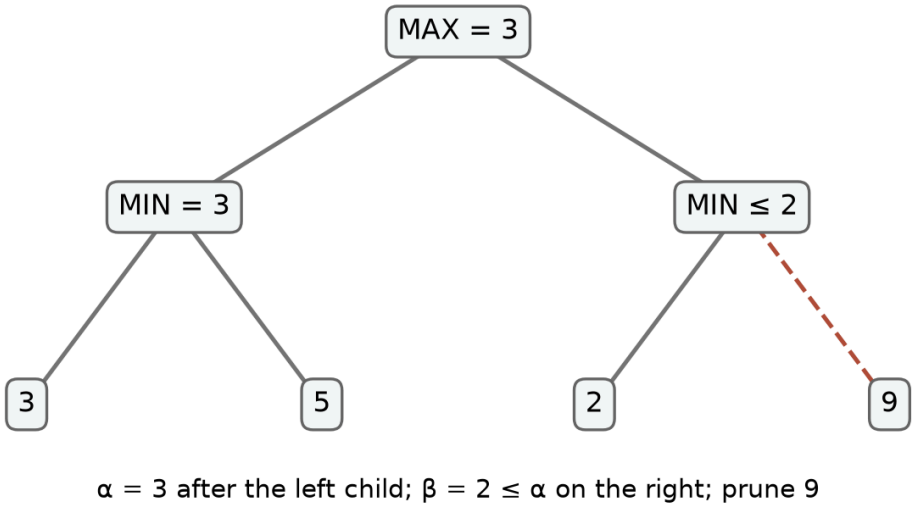


Figure 2.9. Alpha-Beta cutoff on an unchanged tree

Source: Alpha-Beta pruning cutoff formulation [17].

2.10 Iterative deepening and budgets

Iterative deepening performs complete searches at depths 1, 2, … until reaching a configured maximum or a budget interrupts an iteration. Its input is a root position, a maximum depth and a budget. Its output should distinguish the best result from the most recently completed iteration, the completed depth, and the reason for stopping. The working board, search counters, and any table or ordering hints must obey the respective implementation's lifecycle. An incomplete iteration's partially explored best move is not the same as a completed minimax result for that depth.

```
// Algorithm 2.12 (CaroGame/src/game/bot/search.ts) — ITERATIVE(B,
maximum_depth, budget):
    last_complete ← NONE
    for depth from 1 to maximum_depth:
        if budget is exhausted: break
        result ← SEARCH_AT_DEPTH(B, depth, budget)
        if result was interrupted: break
        last_complete ← result
    return last_complete, completed depth, stop reason
```

The invariant after iteration d is that last_complete corresponds to a whole completed search at d under that iteration's search contract. It follows inductively because the result is replaced only after the iteration returns normally. A separate abort-safe restoration guarantee is required: interruption in a child must unwind all applied moves. If depth one returns A, depth two returns B and depth three aborts, the completed result is B at depth two. If no iteration completes, a caller must explicitly handle the absence of a completed result; a fallback candidate is useful but should be labeled as such.

Repeated shallow searches consume work, although the deepest full iteration can dominate for a sufficiently large, regular branching factor. Each iteration has the complexity of its selected search; total time is the sum over attempted depths plus ordering and bookkeeping. No hard real-time guarantee follows from this scheme. A search interrupted only at periodic checks can exceed its nominal wall-clock deadline before the next check, especially if individual operations are expensive.

The legacy TypeScript findBestMove uses performance.now(), a deadline and a periodic node check, and catches its search-abort sentinel. It initializes a fallback from the first candidate before any depth is completed; its completed-depth field

remains zero if the first iteration aborts. Its recursive search wraps move application in try/finally blocks so that an aborted depth unwinds all working-board mutations cleanly [11]. The historical Rust reference instead used a node budget, and its report states that its aggregate omits work inside an aborted iteration. Those semantics should not be combined into one universal time-budget claim [8]. The inspected local game engine specifies fixed depth for its tiers; archived runs separately record each configured depth [18]. Using deterministic fixed-depth Alpha-Beta per tier trades dynamic wall-clock cutoffs for predictable per-tier search depth and reproducible move selection.

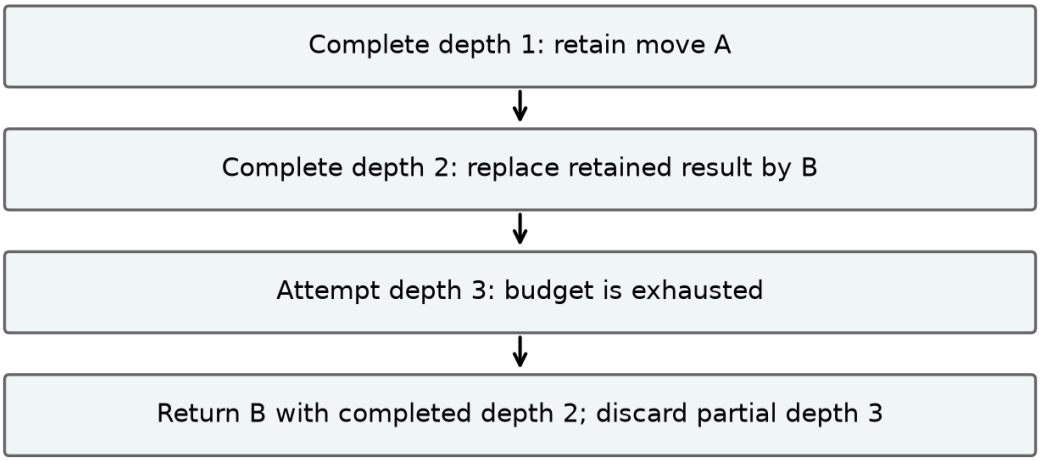


Figure 2.10. Iterative deepening and incomplete work

Source: CaroGame/src/game/bot/search.ts [8].

2.11 Zobrist hashing and incremental state

2.11.1 Definition and update

A transposition can arise when different sequences of moves lead to the same board and relevant side to move. Comparing all cells at each cache access costs $O(N^2)$, so a compact hash can serve as an index. Zobrist's method assigns a pseudorandom word to each (cell, player) pair and combines the keys for occupied cells with exclusive OR (XOR) [19]. One also needs a side-to-move key if the same occupancy can be evaluated with different players to move. The hash is a compact fingerprint, not mathematical proof that two states are identical.

Define Z[x,y,p] as a fixed random word for a stone, and S as a fixed side key. Under one convention,

H(B,p) = XOR Z[x,y,color] over occupied cells, XOR S if p is the designated second side.

All keys must remain fixed for the lifetime of the table. Applying a move of p at m adds its key with XOR. If the transition changes the side, XOR S as well. Undo repeats the same XOR operations in reverse. The essential identity is (h XOR z) XOR z = h.

```
// Algorithm 2.13 (CaroGame/src/game/bot/board.ts) — FULL_HASH(B, player):
    h ← 0
    for each occupied cell m with stone color c: h ← h XOR Z[m,c]
    if player is designated second side: h ← h XOR S
    return h

UPDATE_HASH(h, m, p, side_changed):
    h ← h XOR Z[m,p]
    if side_changed: h ← h XOR S
    return h
```

For an illustrative four-bit example, let h=1010 and the inserted stone's key be 0110, ignoring the side for the moment. Apply gives 1100; undoing XOR with 0110 restores 1010. Because XOR is both commutative and self-inverse, applying the same stone key Z(m,p) and turning key S on both make and undo restores the exact 64-bit state hash in O(1) time.

The hash invariant is incremental_hash = FULL_HASH(current board, current side). It holds initially if computed from the board; XORing the stone's key exactly accounts for adding that feature, and XORing the side key exactly accounts for a side change. Repeating those operations removes the features on undo. This proof assumes the same keys, actual transition semantics, and no omitted state features relevant to the hash contract. It does not eliminate collisions.

The legacy TypeScript board maintains two 32-bit halves, a seeded key table and a side key, and updates them with make and undo. Its pattern counts and stone count are also incrementally updated; restoring only the hash would not restore the whole board state [20]. Historical reference reports document incremental hash checks and note that a terminal winning move may leave to_move unchanged in that separate

implementation, so its side-key update must follow the actual state transition [8]. This distinction cannot be transferred automatically between implementations.

A full recomputation scans $O(N^2)$ cells; one incremental XOR update uses $O(1)$ time. The key table has $O(N^2)$ entries for a fixed number of players; the board and any pattern caches have their own space costs. A finite hash collision remains possible even when incremental and recomputed hashes agree on every tested position. A table may verify a full key or full state, or otherwise accept a defined collision risk. A player key alone also does not encode rules, search depth, evaluator, root perspective or a candidate policy.

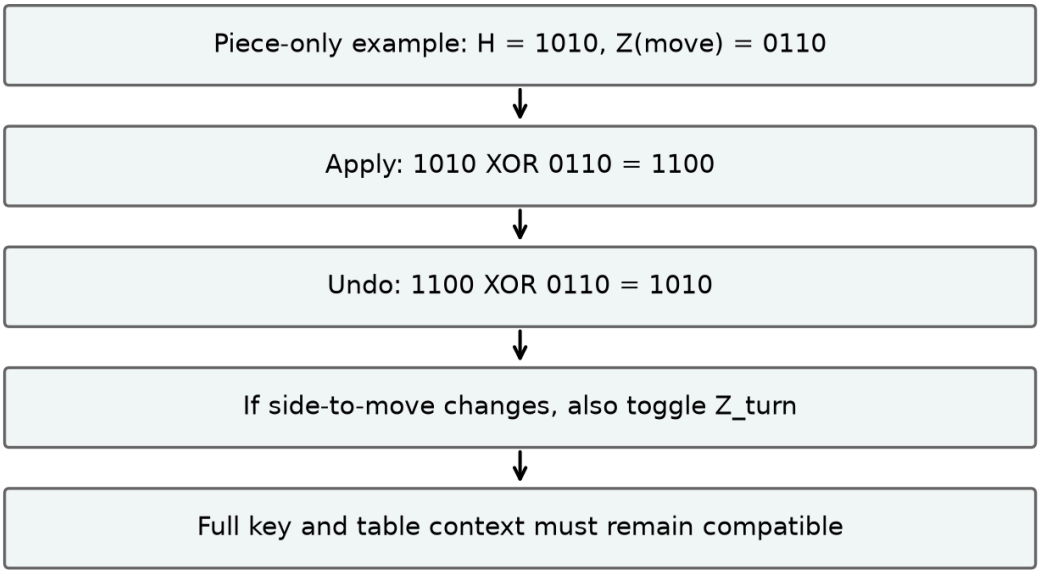


Figure 2.11. Incremental XOR and turn-key handling

Source: CaroGame/src/game/bot/board.ts [20].

2.12 Transposition table: entries, bounds and context

2.12.1 Meaning of stored values

A transposition table (TT) maps a position fingerprint to a previously computed search result. A typical entry stores a hash or verification key, remaining search depth, value, bound type and a suggested best move. The move may help order later search even if the stored score is unusable. A cache lookup is sound only when its represented search problem is compatible with the current one, its depth is sufficient under the selected contract, and its bound justifies the requested cutoff or exact result.

For an original search window $[\alpha_0,\beta_0]$, a common classification is: result $v \le \alpha_0$ is an upper bound; $v \ge \beta_0$ is a lower bound; and $\alpha_0 < v < \beta_0$ is exact, subject to the search's assumptions. Equality is treated conservatively at the boundary. For instance, with window [3,8], a returned 2 is an upper bound, 5 is exact and 9 is a lower bound. Consequently, a stored LOWER bound triggers an immediate cutoff whenever entry.score ≥ β, and an UPPER bound triggers a cutoff whenever entry.score ≤ α.

```
// Algorithm 2.14 (CaroGame/src/game/bot/search.ts) — PROBE(key,
required_depth, alpha, beta, context):
    entry ← table.lookup(key)
    if entry missing or key/context is incompatible: return MISS
    hint ← entry.best_move if it is legal in this position else NONE
    if entry.depth < required_depth: return HINT_ONLY(hint)
    if entry.bound = EXACT: return VALUE(entry.score, hint)
    if entry.bound = LOWER and entry.score ≥ beta:
        return VALUE(entry.score, hint)
    if entry.bound = UPPER and entry.score ≤ alpha:
        return VALUE(entry.score, hint)
    return HINT_ONLY(hint)


STORE(key, depth, score, alpha_original, beta_original, best_move):
    if score ≤ alpha_original: bound ← UPPER
    else if score ≥ beta_original: bound ← LOWER
    else: bound ← EXACT
    store key, depth, score, bound, best_move according to
replacement policy
```

Validating entry depth, bound type, and move legality during table lookup guarantees that cached bounds never corrupt the active Alpha-Beta window. The classification uses the original search window; using a window altered by a prior lookup without carefully defining semantics can mislabel a bound as exact. Cache storage and retrieval must agree about whether a returned cutoff value is fail-hard or fail-soft.

2.12.2 Why reuse can be wrong

A useful invariant is: whenever a TT score is reused, it is valid for the current position, depth interpretation, perspective, rule set, evaluation and move-generation context, and its bound permits the intended conclusion. An exact compatible entry may answer a query; a lower or upper bound is usable only in its permitted direction. If any premise is false, the stored number can change the decision. A full hash comparison narrows ordinary index collisions but still does not guarantee that different game states cannot share a fingerprint.

The historical reference report documents a concrete context hazard. It records one same-root warm reuse, then a probe after playing a move and changing root player: a fresh table returned score 0, whereas reuse of the prior table returned 19,800. Its hash omitted several contextual factors, including root perspective, and the report does not claim a patch. These are results of a particular native reference probe, not a measurement or defect proven in the current browser engine [21]. The report also notes that mate scores depend on ply from the root; a cached value reused at another ply may require normalization or a sufficiently narrow reuse scope. Rules, evaluation weights, side, candidate radius/cap, ordering semantics, terminal metadata and requested depth can likewise matter. A board fingerprint by itself does not certify those conditions.

The legacy TypeScript uses parallel fixed-size arrays indexed by a modulo operation on one hash half and compares both halves before using a slot. It stores depth, score, flag and best move. This is a different storage and replacement design from the reference's hash map [11], [8]. The existence of a depth check does not resolve every context hazard; in particular, project-wide claims require inspecting lifecycle, perspective and score conventions. A cached move hint should be checked against current legal candidates before application. A new game, changed rules, new evaluator or changed root is a reason to reassess or clear a cache unless compatibility is proved.

Expected lookup can be O(1) for an appropriate hash table, but collisions, table implementation and replacement behavior affect actual costs. Space is proportional to the allocated slots or stored entries plus keys, buckets and overhead. Initial hash-map capacity is not a strict maximum, and the native size of one entry is not the total memory of the container. Benefits and trade-offs of transposition tables: O(1) expected hash lookup avoids re-searching transposed subtrees and supplies best-move ordering hints, at the trade-off of O(M) table memory, potential 64-bit

hash collisions, and the requirement to scope or clear cached entries when root perspective or rule context changes.

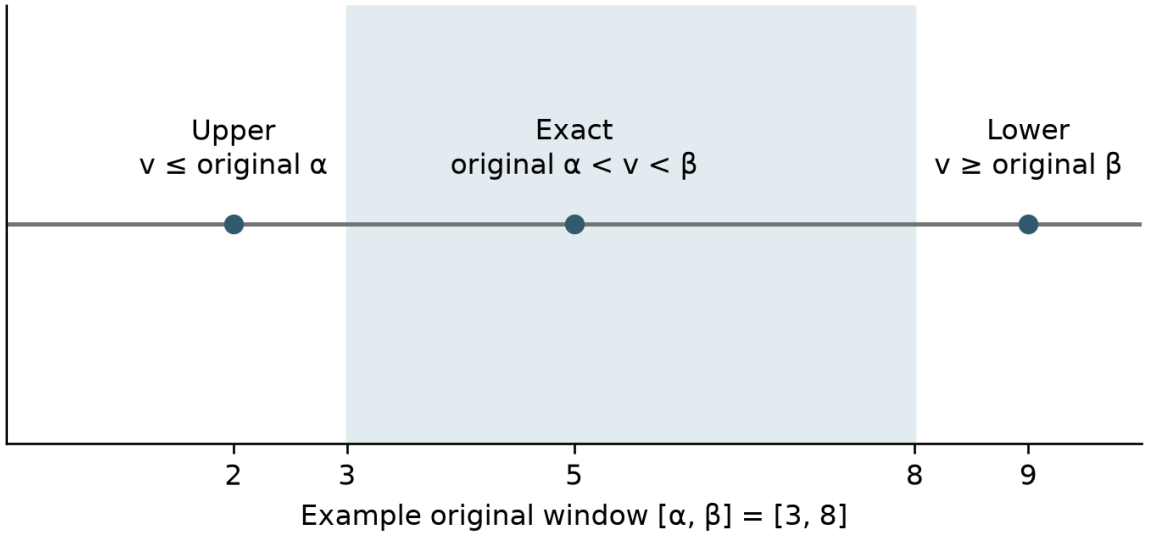


Figure 2.12. Bound semantics in a transposition table

Source: CaroGame/src/game/bot/search.ts [8].

2.13 Quiescence search in the legacy TypeScript bot

A fixed-depth search can stop at an unstable position: a static evaluator may prefer a move just before an immediate tactical reply. Quiescence extends selected tactical continuations beyond the ordinary horizon so that the evaluator is applied after a limited forcing sequence. This is a selective model and has its own horizon; it cannot eliminate every tactical error. The relevant implementation is the legacy TypeScript bot, not a feature established for the current WASM engine [11].

Inputs and state. The legacy routine receives the board, alpha and beta bounds, current ply and a quiescence depth. It works from the perspective of the side to move, uses negamax sign reversal, increments a node counter and checks for abort. Its state includes board cells, side, hash and cached pattern counts. Its output is a bounded score for the selected forcing continuation, or an abort propagated to the caller. It first checks whether either side has a five pattern. It then calculates standPat, applies a beta cutoff if that static score is at least beta, raises alpha if appropriate, and stops at quiescence depth five. Otherwise it generates forcing candidates, orders them and recurses after a move with (-beta,-alpha); each recursive branch uses try/finally to undo [11].

```
// Algorithm 2.15 (CaroGame/src/game/bot/search.ts) — QUIESCENCE(B,
alpha, beta, ply, qDepth):
    count node; check budget or deadline
    if previous player has a five: return -MATE + ply
    if side to move has a five: return MATE - ply
    standPat ← EVALUATE_FROM_SIDE_TO_MOVE(B)
    if standPat ≥ beta: return beta
    alpha ← max(alpha, standPat)
    if qDepth ≥ 5: return alpha
    moves ← GENERATE_FORCING_CANDIDATES(B)
    order moves by their quick score
    for m in moves:
        MAKE(B,m)
        try: v ← -QUIESCENCE(B,-beta,-alpha,ply+1,qDepth+1)
        finally: UNMAKE(B,m)
        if v ≥ beta: return beta
        alpha ← max(alpha,v)
    return alpha
```

The sign change reflects the negamax convention for an evaluation whose perspective changes with the side to move. The procedure returns beta on a fail-high and alpha otherwise, so its return can be a bound rather than an exact unrestricted game value. Its loop invariant is that alpha is at least the stand-pat score and all examined forcing continuations, subject to the window and fail-hard convention; the board must equal the incoming state before every sibling. The depth limit guarantees termination even when forcing candidates continue to exist. It also creates a new horizon.

For a simplified numeric example, suppose the window is [−10,10], stand-pat is 2, and the first searched forcing move returns 6 after sign reversal. Alpha rises to 6. If another move returns 12, the routine returns beta, 10, as a fail-high bound. This does not assert that 10 is the exact minimax value. In a real position, the forcing generator's definition and threat patterns decide which moves are included. The legacy code considers outcomes such as five, open four and four for either side when identifying forcing moves. These categories arise from its own bitmask

pattern evaluator, which differs from the snapshot's contiguous-run evaluator [11], [20].

The stand-pat assumption deserves care in a game where a player must move: static evaluation does not literally grant a legal pass. In a forced-defense position an immediate static cutoff may not establish the same value as examining all legal replies. The finite depth, selective forcing set and candidate implementation further limit the result. A rigorous claim requires a specified tactical contract and tests that compare against an appropriate full or deeper reference on representative positions. With at most f forcing moves at each quiescence level and maximum extra depth q, a coarse upper bound is O(f^q) nodes plus candidate generation, ordering and evaluation at each node; actual cost may differ sharply. The stack and stored move lists take O(qf) space under the same conventional assumption, in addition to board and caches.

2.14 Legacy bitmask patterns and incremental line counts

The older TypeScript bot represents a line segment available to one player as a bitmask: a one denotes the player's stone and zero denotes an empty cell; an opposing stone divides the line into separate segments. It precomputes a classification for eligible lengths and masks using completion tests. Categories include a five, an open four, a four, an open three and a broken three. The terminology is the implementation's category scheme; it must not be silently equated to the maximal contiguous runs and weights of the documented Rust snapshot [20], [11].

```
// Algorithm 2.16 (CaroGame/src/game/bot/board.ts) —
CLASSIFY_SEGMENT(mask, length):
    if mask contains five consecutive 1 bits: return FIVE
    wins ← number of zero positions which become a five when set
    if wins ≥ 2: return OPEN_FOUR
    if wins = 1: return FOUR
    threats ← number of zero positions which, when set,
              leave at least two immediate winning positions
    if threats ≥ 2: return OPEN_THREE
    if threats = 1: return BROKEN_THREE
    return NONE

COUNT_LINE(B, p, start, step, length):
    split the line wherever an opponent stone occurs
    encode p and empty cells of each segment as a mask
    for each segment of length at least 5:
        increment the bucket returned by CLASSIFY_SEGMENT
    return counts
```

For a hypothetical five-cell segment XX_XX, setting the middle zero creates five consecutive bits, so the segment has at least one immediate winning completion under this rule. A segment with two distinct empty completion cells is classified above a one-completion segment, assuming no five is already present. The precise category depends on the whole segment and the code's precedence, not only on a local substring. Opponent cells split the segment; a board boundary is also a boundary. Short segments cannot contain five and do not contribute a five-capable pattern.

The board's incremental update subtracts old counts for the four lines crossing a changed cell, makes the move, then adds counts for the new versions of those lines. Undo performs the inverse around clearing the cell. The invariant is that accumulated per-player counts equal a fresh sum of classifications for the board, assuming the untouched lines are unchanged and the four affected lines are correctly identified. Each changed cell lies on exactly one line of each axis, so only

those lines need recomputation. This is a proof sketch for the update scheme, not a proof that the classifier expresses every strategically meaningful formation.

The legacy position evaluator shown in the repository scans lines and sums category weights from the selected perspective; cached counts are also used by other routines. A full evaluation over a constant number of axes is $O(N^2)$ for fixed-time table lookup per segment; recalculating four affected lines is $O(N)$ in board width per player, in addition to the cost of table lookup. Precomputing a pattern table trades initialization work and memory for later lookup; its precise limits, size and supported board lengths are implementation constraints. The actual code initializes for line lengths up to 15, so changing board size without reviewing this bound is unsafe. A cached count is only trustworthy when every mutation follows the update discipline and every aborted sequence restores it.

2.15 Difficulty configurations and the active request path

The application partitions gameplay into two primary modes—local two-player PvP (1v1) and human-versus-bot PvE (ai)—and divides the PvE bot into six progressive difficulty tiers (Easy, Normal, Hard, Very Hard, Expert, and Master), whose strategy, search depth d, Chebyshev radius r, and soft candidate cap c are specified in Table 2.1 [18]. Each tier maps to an explicit search strategy and candidate-generation policy in the Rust engine dispatch [6]. The TypeScript WASM contract validates and forwards the selected difficulty identifier and returned telemetry across the Web Worker boundary [9].

Table 2.1. Difficulty configurations and recorded search behaviour

Difficulty	Recorded strategy	Depth	Radius / soft cap	Interpretation
Easy	Random	0	All legal empty cells	Bypasses tactical prepass; selects uniformly via seeded PRNG (no look-ahead).
Normal	Rule-based	1	Neighborhood, no fixed cap in default policy	Bypasses frontend prepass; Rust 1-ply immediate win/block then static heuristic.

Difficulty	Recorded strategy	Depth	Radius / soft cap	Interpretation
Hard	Alpha-beta	2	2 / 15	Frontend tactical prepass first; 2-ply Alpha-Beta over up to 15 local candidates.
Very Hard	Alpha-beta	3	2 / 12	Frontend tactical prepass first; 3-ply Alpha-Beta (API ID 5) with soft cap 12.
Expert	Alpha-beta	4	2 / 10	Frontend tactical prepass first; 4-ply Alpha-Beta with soft tactical cap 10.
Master	Alpha-beta	5	2 / 8	Frontend tactical prepass first; 5-ply fixed-depth Alpha-Beta with soft cap 8.

Why the six difficulty tiers are divided this way: the configuration table is designed around two mathematical principles. First, it separates qualitative decision regimes: Easy (d = 0, Random across all empty cells) provides a non-tactical baseline for beginners; Normal (d = 1, uncapped local neighborhood) introduces 1-ply immediate win/block priority and static heuristic scoring; and Hard through Master (d = 2 to 5, Chebyshev radius r = 2) engage multi-ply Alpha-Beta look-ahead. Second, across the four Alpha-Beta tiers, search depth increases linearly (d = 2, 3, 4, 5) while the soft candidate cap tightens inversely (c = 15, 12, 10, 8). Because an adversarial tree with branching factor c and depth d scales as O(c^d) in the worst case (and O(c^(d/2)) under ideal Alpha-Beta ordering), keeping c = 15 at d = 5 would expand up to 15^5 = 759,375 nodes per turn. Tightening the soft cap to c = 8 bounds the worst-case tree to 8^5 = 32,768 nodes (and ~2,000–5,000 nodes with Alpha-Beta pruning), keeping Web Worker latency within interactive browser budgets (47, 252, 551, and 1,418 visited nodes in 1.1–3.0 ms on the benchmarked midgame fixture). Benefits and trade-offs: this inverse (d, c) scaling delivers smooth skill progression and responsive UI performance while the soft tactical cap max(c, 1, |T|) and frontend tactical prepass guarantee that zero immediate 1-ply wins or blocks are ever dropped; the trade-off is that deeper tiers (Expert, Master)

prune lower-ranked quiet positional candidates (ranked below index 10 or 8) in exchange for deeper 4-to-5-ply tactical calculation.

The current frontend controller's input is an application state, round identity and difficulty. It increments a request identifier. For Easy and Normal it proceeds to the WASM engine adapter. For the four higher tiers it first asks the TypeScript tactical resolver for a forced move for O; if one is returned, it creates a tactical result without requesting engine search. Otherwise it copies move history into a request to the engine. The frontend prepass and the Normal Rust rule-based policy are different decision stages [10].

The archived adapter excerpt shows a new game instance being populated from the request's move history, rejecting an unsuccessful replay, asking for an AI move, freeing the game in finally, converting Rust (x,y) into frontend (row,col), and attaching request/round identity and statistics. Its elapsed scope is named worker-hydrate-and-search, which includes history reconstruction and the search call, rather than being an isolated inner-loop benchmark [6]. By pairing Worker-level hydration and cleanup with controller-level requestId and roundId verification, the architecture guarantees that stale asynchronous search completions from prior turns, undos, or resets are deterministically discarded before mutating the active board.

2.16 Synthesis and limits of the algorithmic claims

The algorithms in this chapter have different responsibilities. Win detection decides the stated terminal rule; board mutation maintains a position; random and rule-based choices provide baselines; the evaluator assigns an approximate horizon value; candidate generation restricts the moves explored; ordering permutes the retained moves; minimax evaluates the resulting tree; alpha-beta skips provably irrelevant branches of that same tree; iterative deepening manages progressive search; and hashing plus a transposition table aim to reuse compatible earlier work. Quiescence extends a selected class of tactical moves at a horizon, but its own assumptions and depth limit remain.

Each algorithmic layer enforces a distinct invariant within the overall decision pipeline: 4-axis ray scanning detects Freestyle terminal lines in O(K) time; in-place apply/undo preserves board state across recursive branches in O(1) time; Chebyshev spatial generation and the soft tactical cap bound the branching factor while retaining 100% of immediate 1-ply threats; move ordering prioritizes high-value branches; Alpha-Beta pruning eliminates provably suboptimal subtrees

while preserving the exact Minimax root value; and Zobrist XOR hashing with bound-typed transposition tables accelerates repeated state lookups.

The recorded research tests and experiments are useful because they specify fixtures, configurations and counters, including negative cases and restoration checks. Together, the native algorithmic benchmarks, Node WASM parity checks, and browser Web Worker telemetry provide complementary multi-level validation [8]. For a reproducible claim about a particular application build, the report must identify the source revision or snapshot, binary provenance, rule and evaluation configuration, player and position, candidate set, depth, platform, and meaning of each reported counter. The following chapters can then describe the system integration and evidence without turning theory into an unmeasured product outcome.

CHAPTER 3. SYSTEM ANALYSIS AND DESIGN

3.1 Actors, use cases and functional requirements

The primary actor is a person using the web application. The person can play against another person on the same board or select a computer opponent. The AI service is a system component rather than an independent user: it receives a snapshot and returns a proposed move. A hosting integration may supply lifecycle information, but it does not own the board or bypass the controller's move checks. This actor model keeps responsibility for accepted state transitions inside the application. [2] [22]

Table 3.1. Functional requirements F01–F09

Requirement	Trigger and input	Required behaviour and observable result
F01 Select mode and difficulty	A person selects local two-player or AI mode, or an available AI difficulty.	Invalidate the previous AI turn, reset board and search feedback, establish the selected mode and setting in a ready match, and carry the selected difficulty in subsequent AI requests.
F02 Validate and place a stone	A pointer gesture resolves to a board coordinate or another attempted move occurs.	Accept one legal current-player move to an in-bounds empty cell, update history and evaluate the outcome. Reject occupied or out-of-bounds moves, paused or ended games, a locked AI turn and wrong-player moves without changing the board.
F03 Determine the round outcome	An accepted move completes a Freestyle winning line or fills the board without a win.	Check the four axes, record and display a win or draw, and prevent further moves in the ended round.
F04 Maintain history, undo and reset	A move is accepted, or a person invokes Undo, Replay or New Game.	Maintain the move history. Undo one ply in two-player mode or the human move and a completed AI reply where applicable, restoring the correct turn and phase. Replay and New Game reset board and feedback, invalidate the old AI request and establish the phase intended by each action.

Requirement	Trigger and input	Required behaviour and observable result
F05 Request and validate an AI move	An accepted nonterminal human move changes the turn to the computer in AI mode.	Request one proposed move, lock board input while the computer is thinking, and commit only a still-current legal result; invalidate responses belonging to an earlier round or request.
F06 Show AI feedback and retry	An AI request is pending, succeeds, or fails.	Display thinking state and, when supplied, strategy, source, counters, elapsed scope or an error. Permit retry when provided by the application; do not invent Rust statistics for a route that did not run Rust.
F07 Render and resize the board	Board state changes or the available drawing region changes size.	Render the board and stones from the current state and update the board view for the changed drawing region.
F08 Host integration and standalone operation	A host supplies locale or changes pause state, or the game is opened standalone.	Apply supported host locale and pause signals when integrated, remain usable in standalone mode, pause the implemented game-owned input and rendering activities and invalidate stale AI work, and resume a needed AI turn from current state.
F09 Inspect presentation material	A reader uses a separate presentation site.	Provide explanatory material, figures and the Python examples available in that site, keeping instructional demonstrations separate from engine benchmarks.

The requirements are derived from controller actions and their guards, not merely from the presence of buttons. For example, replay and new game both clear stones, but they have different phase semantics. A reset-like visual action must be traced to the method it calls before its behaviour is described. Similarly, removing search feedback after Undo is part of keeping displayed evidence associated with the current board. [2] [23]

Completion and verification criteria. The following criteria define the expected result for each functional group; they are acceptance targets, not a claim that every scenario has already passed. The detailed evidence and its environment are given in Chapter 5.

F01: Switching between two-player and AI modes or choosing a difficulty starts the intended match configuration and invalidates an earlier AI request; the selected tier is carried into subsequent requests. Verify each selection and the board/turn state before and after the change. F02: A valid move has integer row and column in 0–14, targets an empty cell and is made by the permitted player in an active, unlocked round. Accept it once and change one cell; reject occupied, out-of-range, fractional, paused, ended, wrong-turn and AI-locked inputs without mutating the board or history. Both the pointer coordinate transform (via Math.floor) and the WASM adapter (via Number.isInteger) enforce integer grid indices in 0–14 before any move reaches the board state.

F03: In each of the four axes, test a five-stone line and an overline of six or more, including a boundary example; both must win under Freestyle. A full board without a winner must draw, while a win on the last cell must take precedence over a draw. F04: Verify the documented Undo behavior for two-player and AI play, restoring cells, history, turn and phase; Replay/New Game must reset the intended state and invalidate old work. F05: During an AI turn, lock input and accept only a legal reply for the active request and round; test late replies after Undo, Reset, mode change and cancellation. F06: Show pending, result and error/retry states as applicable; attribute strategy, counters and elapsed scope to the route that produced them, leaving unavailable measurements absent.

F07: On a state update or viewport resize, render the current board and keep coordinate mapping correct. F08: Check locale and pause/resume behavior in an actual supported host and standalone mode, including stale-response handling and a needed AI restart; do not infer host acceptance from unit tests. F09: Open the presentation figures and Python examples independently, and label demonstrations as teaching material rather than live-engine performance results.

For algorithm verification, compare Minimax and Alpha-Beta on identical positions, rules, root perspective, depth, evaluation and candidate membership; equal root scores are required, with tied moves allowed to differ. The native test suite verifies exact score agreement across 408 Minimax/Alpha-Beta comparison pairs and 820 root-restoration checks, while Node parity tests and headless Chrome Worker tests verify the compiled WebAssembly binary and asynchronous controller guards end-to-end.

3.2 Non-functional requirements and acceptance boundaries

Correctness means preserving legal state transitions, the agreed rule and the relation between board cells and history. Maintainability means keeping rule evaluation, rendering, input conversion and asynchronous search behind identifiable interfaces. Responsiveness motivates the Worker boundary for substantial search, while correct cancellation requires more than moving work off the main thread. Reproducibility requires a source revision, binary identity, configuration and measurement procedure rather than a screenshot containing a duration.

Executing the Rust/WebAssembly search inside a dedicated Web Worker isolates multi-ply tree expansion from the main browser thread, preserving responsive PixiJS canvas rendering and deterministic request cancellation across all six difficulty tiers.

3.3 Architecture and ownership

The system has five principal responsibilities. React owns interface composition and subscribes to a controller snapshot. GameController owns the game-facing action boundary, current-player choice, AI-turn lifecycle and notification. The core owns board cells, legal-move history and rule evaluation. Pixi owns the visual board and converts gestures through an input/coordinate layer. The AI adapter owns search dispatch and Worker resources; it proposes a move but does not directly mutate the authoritative application board. [22] [2] [24] [14]

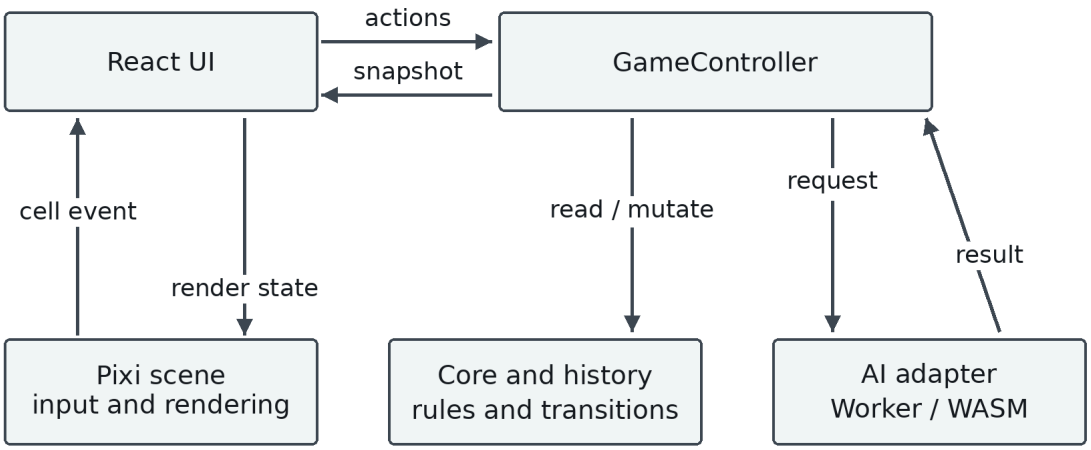


Figure 3.1. Responsibilities in the playable application

Source: CaroGame/src/app/GameController.ts [2].

This ownership arrangement gives a clear dependency direction. The scene reports a cell action to the application/controller. The controller changes domain state and publishes a snapshot. The scene receives a list of stones, last move and winning cells for rendering. A visual object is not a second authoritative board. The AI path receives copied history and returns a result through the controller, which repeats the relevant validity checks before committing it.

A snapshot contains the match record, current player, thinking and pause flags, board stones, history, last move, last AI result and error. The controller copies move records and their cell coordinates when creating it. Subscribers read a stable cached snapshot until a notification creates a new one. This matters for React subscriptions: returning a newly constructed object on every read could produce unnecessary updates or unstable identity, whereas mutating a previously delivered object could conceal a change. The observed implementation uses an explicit cached value and listener set. [2]

Web Workers provide a standard way to execute scripts independently from interface scripts, but that standard does not automatically establish this project's lifecycle safety. Here the Worker wrapper is responsible for transport and pending promises, while the game controller is responsible for deciding whether a result remains valid for the current round. These are complementary responsibilities.

Terminating a worker can stop obsolete work; checking the current token prevents an already-resolved obsolete response from being committed. [25] [26]

3.4 Board state, history and legal transition contract

The frontend GameState stores one value per cell and an ordered move history. A move records its player, cell and index; history is also the input used to reconstruct a fresh Rust game for a search. These two uses require consistency. If a board cell changed without a corresponding history update, the display could show one position while the engine searched another. The transition contract therefore relates the occupied-cell set to the history sequence rather than treating them as independent data. [14]

The board has $N = 15$ and $N^2 = 225$ cells. A frontend cell is (row, col); Rust entry points use (x,y) with x = col and y = row. The row-major linear index is row × N + col. The normal pointer route applies floor, producing integer cell coordinates, and checks that they lie in 0 through 14. The WASM result adapter separately validates both integer-ness and bounds before mapping a result to a frontend cell. The core isBoardCell function itself checks numeric bounds but does not test Number.isInteger; direct fractional inputs therefore require an additional guard. A public API should not rely exclusively on the renderer to supply integers, because tests or other callers can enter through other routes. [27]

The playMove behaviour rejects an occupied cell before mutation, records the accepted move and evaluates the supplied rule. The higher controller supplies the current player and enforces pause, phase and AI-turn guards. This division is significant: a low-level board operation may not know which UI phase is active, while the controller should not duplicate the geometric win scan. The report therefore describes the legal move as a composed contract rather than attributing every guard to one function. [28] [2]

The history sequence is not merely a move counter. It preserves alternation and the exact order needed for hydration. A full board without a winner is declared a draw after the win test. That order matters on the last empty cell: a winning final placement must remain a win rather than being classified as a draw merely because the board is now full. The controller's finishMove uses this ordering. [2]

3.5 Match phases and state transitions

MatchController defines setup, ready, playing and game-over. Its initial state is ready, local two-player mode, normal difficulty and round identifier zero. Calling

playCell from setup or ready invokes startRound, which increments the round identifier, clears the old outcome and enters playing. An accepted win or full-board draw changes the match to game-over. The thinking flag belongs to the AI request lifecycle; it is not a fifth value in the phase enumeration. [23] [2]

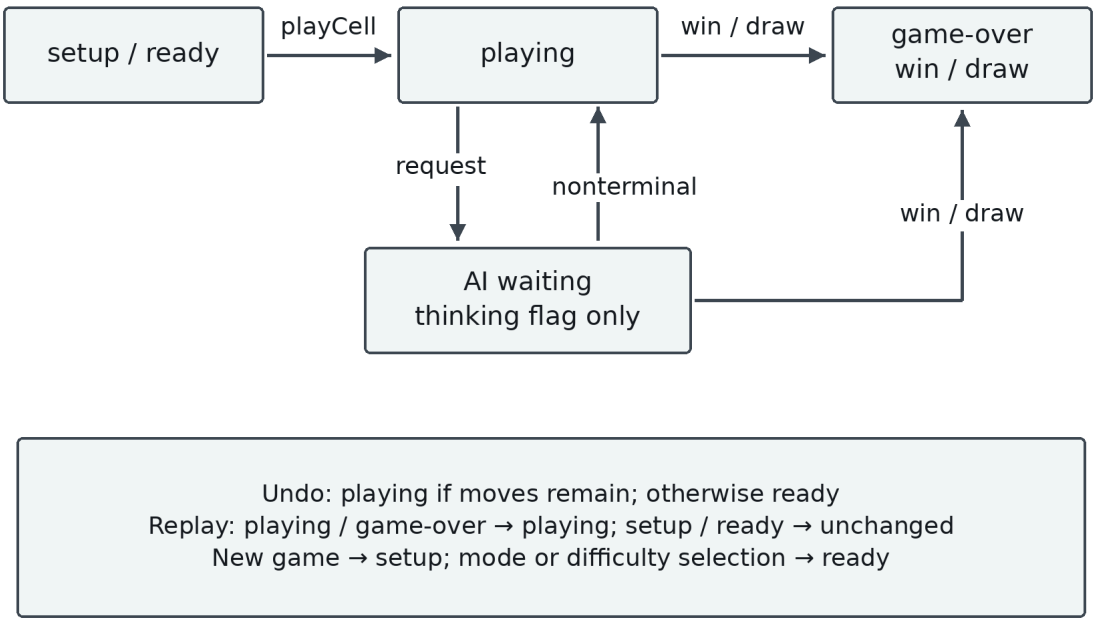


Figure 3.2. Match phases and the separate AI waiting flag

Source: CaroGame/src/game/behaviors/match/MatchController.ts [23].

New game increments the round identifier and enters setup. Selecting a mode or difficulty first clears old game work and then places the match in ready. Replay may enter playing directly when the previous phase is playing or game-over. Undo calls resumeAfterUndo, increases the round identifier and chooses playing if stones remain, otherwise ready. These distinctions explain why a state diagram must name actual events instead of using a single unspecified Reset arrow.

Pause is orthogonal to the match phase. A paused playing round remains a playing round whose input and current AI work are suspended or invalidated. On resume, the controller checks whether it is still an AI match, still playing and still O's turn before requesting again. If pause were represented as a new board with an implicit player reset, the resumed game could diverge from its history. The source keeps the board and phase while handling the lifecycle separately. [2]

A state-machine invariant can now be stated. A result may commit only when its request belongs to the active AI turn and round, the application is unpaused, the

phase is playing, the mode is AI and O is the current player. None of these predicates is replaced by the fact that a promise resolved successfully. Success describes the completed computation; the predicates describe whether that computation is still applicable.

3.6 Asynchronous request protocol and cancellation

An AI action captures a new turn token, the round identifier and difficulty before setting thinking state. The request contains a copied move history. The transport wrapper adds its own message identifier so the correct pending promise receives the corresponding Worker response. The returned domain result carries its request/round identity, difficulty, move and statistics. The message identifier, AI request identifier and round identifier serve related but different scopes; they should not be collapsed into a single undocumented counter. [2] [3] [26]

On completion, the controller compares the captured token with its current token, checks pause and the returned round identifier, and repeats the phase, mode and current-player conditions. Only then does it clear thinking state, save feedback and pass the proposed O move through the same domain move behaviour used elsewhere. A rejected returned move produces an error instead of being painted directly. A missing move in an active round is also an error. This keeps the authoritative board inside the domain path. [2]

A concrete race illustrates the mechanism. Suppose request A is searching round 8. The person invokes a New game before A finishes. The controller increments its token, clears state and changes the round. Even if A's response was already queued, its captured token no longer matches. The promise resolution cannot insert A's move into the new board. Meanwhile, cancelPending rejects pending transport promises and terminates the old Worker, so future requests construct a new one. The token guard and resource cancellation address different parts of the race. [2] [26]

The rejection path is also scoped. A caught error changes feedback only when the captured token still equals the current token. Thus cancellation of an old request should not overwrite the error display of a newer turn. The Worker wrapper ignores messages from a worker object that is no longer its current worker, and ignores message identifiers absent from its pending map. These guards are source mechanisms; whether all possible races are covered remains a test question.

3.7 Coordinate transformation and gesture interpretation

The scene works with a camera translation (cx,cy), a positive zoom z and cell size S = 56. For a screen position expressed in the scene's coordinate system, the inverse transform is worldX = (screenX − cx)/z and worldY = (screenY − cy)/z; the selected column and row are floor(worldX/S) and floor(worldY/S). Conversion back to a cell's world origin multiplies its column and row by S. A centre point for drawing a stone is obtained by adding half a cell in the renderer, rather than changing the indexing contract. [29]

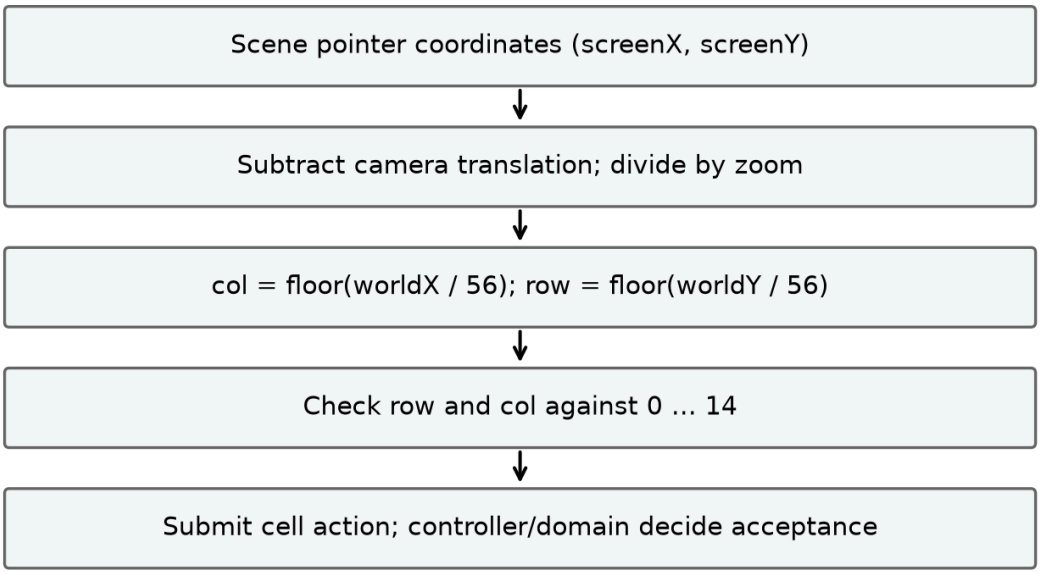


Figure 3.3. Inverse camera transform before board bounds checking

Source: CaroGame/src/board/spatial/CoordinateTransform.ts [29].

The floor operation is important near boundaries. A negative world coordinate must remain out of bounds; rounding it toward a nearby valid cell could turn a margin tap into a move. For example, with camera x = 20, zoom = 0.5 and screen x = 19, the world x is −2 and the column is −1. The subsequent bounds check rejects it. With screen x = 230, world x is 420 and the column is 7. The same calculation on y yields the row. The report uses scene screen coordinates; client-page coordinates must first be converted by the event/rendering system that supplies them.

The input controller has idle, pressed and dragging states and a configured drag threshold of six screen units. A press stores the last pointer. Each move computes displacement from the last stored pointer; a sufficiently large displacement while pressed changes the state to dragging, and dragging produces a pan delta. On

release, a valid cell tap is emitted only if the gesture remains pressed. The last-pointer update means the implemented threshold is checked against the last event displacement, not a guaranteed accumulated distance from the initial press. This detail is preserved here so the explanation does not promise a different gesture recognizer from the source. [30]

The scene treats pointer-up outside through the same handler and suppresses event processing while paused. It fits the board to the viewport using the minimum of available width and height ratios, with different padding for a narrow viewport. The camera's translation and scale are then applied to the world container. This separation lets the same rule and history logic work under different viewport sizes, although a desktop viewport simulation does not establish physical-touch behaviour. [24]

3.8 Search-result contract and presentation of telemetry

The Rust result validator expects integer x/y coordinates in bounds, a finite score, the requested difficulty, one of the known strategy names and nonnegative safe-integer counters. Those counters include visited nodes, leaf evaluations, cutoffs, ordering evaluations, generated candidates and completed depth. An older binary returning only a coordinate pair fails this contract. This is preferable to silently displaying a nominal depth beside a result whose engine work was never instrumented. [27] [9]

The adapter maps the fields into application terminology and explicitly labels the source as Rust. Its elapsed timer starts before construction and history hydration, includes ai_move and free, and ends after cleanup. It excludes module loading and message delivery. A displayed duration therefore has a defined scope: it is not a complete user-perceived round trip and is not a measurement of only the recursive search. The tactical route instead labels its source as tactical, reports zero Rust search counters and a null score. Zero counters on that route mean Rust search was bypassed, not that a nontrivial Rust tree required no work. [3] [27]

Completed depth is the depth reported for the finished computation. A configuration table is a requested policy, while completed depth is output telemetry. Neither alone proves that a higher depth is supported or that every node at that depth was examined under a full-legal move set. Candidate pruning, terminal leaves and the chosen strategy all affect the actual search. The interface should preserve these distinctions when showing details to a player or evaluator.

3.9 Verification scenarios derived from the design

Table 3.2. Verification scenarios and expected outcomes

Scenario	Input/event sequence	Required observation
S01 Legal placement	Tap an empty in-bounds cell while the current turn is available	One history item and stone added, correct outcome and next player.
S02 Rejected placement	Tap an occupied or out-of-bounds cell	No board/history mutation.
S03 Last-cell win	Place the final empty stone so it completes a winning line	Win is recorded before a possible draw condition.
S04 Delayed AI response	Request AI, change the round, then deliver the old response	Old result cannot commit a move or overwrite current feedback.
S05 Undo before reply	Human move followed by Undo while a request is pending	Request invalidated and board/turn restored consistently.
S06 Undo after reply	Completed human-plus-AI pair followed by Undo	Correct number of plies removed and feedback cleared.
S07 Worker failure	Worker raises an error, followed by a valid retry	Pending work rejects; error visible; a new Worker can serve a new request.
S08 Pause/resume	Pause during O's AI turn, then resume	Input is suppressed; obsolete work discarded; needed current turn can restart.
S09 Resized scene	Change viewport size after board content exists	Transform and visible board update without changing domain cells.
S10 Telemetry mismatch	Use a result missing counters or with invalid coordinates	Adapter rejects the incompatible result rather than inventing fields.

These scenarios are acceptance statements derived from ownership and invariants. Chapter 5 identifies the subset supported by the published test/observation artifacts. A scenario's presence in this table does not assert that a browser, touchscreen or hosting platform has passed it.

CHAPTER 4. IMPLEMENTATION

4.1 Toolchain and project organization

The application package declares React 19, React DOM, Pixi 7.4, TypeScript, Vite, Vitest, i18next and related integration packages. Its scripts provide development serving, production build, preview, formatting, static typechecking and a test run. Deterministic dependency resolution is enforced via package-lock.json and Cargo.lock across the frontend and Rust workspaces. [31]

The source folders reflect the design boundaries. app contains the game controller. game/core supplies state, bounds, coordinates and rules. game/behaviors implements move and match transitions. game/ai defines the service and tactical route, while game/wasm provides the worker, adapter, typed contract and generated engine package. board/input, board/spatial and board/render separate gestures, transforms and drawing. pixi integrates them into a scene; React components provide controls and feedback around the canvas. [1]

Within CaroGame/src, the modular separation between domain logic (game/core, game/behaviors), asynchronous AI dispatch (game/ai, game/wasm), and hardware-accelerated rendering (board/*, pixi/CaroScene.ts) keeps the core rules decoupled from the presentation layer. [31] [24] [32]

4.2 Accepted move processing

At the action boundary, GameController.playCell first rejects a paused state. It starts a round when appropriate, rejects an ended match and prevents a human action while AI is thinking or while O owns the turn in AI mode. The controller then calls playMove with the current player and proposed cell. A nonaccepted result returns immediately without modifying board or history. However, from setup or ready, startRound runs before cell validation: a rejected direct call can advance the internal phase and round identifier without a notification. Normal pointer input filters out-of-bounds cells before this call; stronger atomic rejection at the controller boundary remains a verification and improvement item. For an accepted result, it processes a win or draw, otherwise alternates the player, notifies subscribers and starts AI only when the match is still playing in AI mode with O to move. [2] [28]

The sequence is deliberate. Notification occurs after the accepted domain change, so the interface receives a coherent snapshot. The AI request is based on the updated history, not on a speculative screen state. Rule evaluation occurs through the supplied FreestyleGomokuWinRule. The scene never paints a permanent stone

merely because a pointer event arrived; it renders the state returned through the controller. This permits the same move contract to reject an occupied cell consistently in the UI and in tests.

Undo is implemented by history reversal. In local two-player mode it removes one ply. In AI mode, if the last recorded player is O, it removes two plies; otherwise it removes one. Before reversal it invalidates AI work and clears search feedback. It then determines the current player from the remaining last move, asks the match controller to resume after Undo and notifies. Popping two plies when the AI has already replied returns the board directly to the human player's decision turn while preserving chronological history integrity. [2]

4.3 AI dispatch, Worker execution and WASM hydration

AIController.requestMove assigns a request identifier and inspects the difficulty. Easy and Normal bypass the TypeScript tactical shortcut so they actually execute the Rust Random and Rule-based policies. Higher tiers ask the tactical resolver for an immediate forced action for O. If one exists, the controller resolves a typed result with tactical source, null score and explicitly zero Rust counters. Otherwise it copies the entire accepted history and delegates to the WASM engine wrapper. This distinction is visible in the result contract and should remain visible in experimental interpretation. [3] [10]

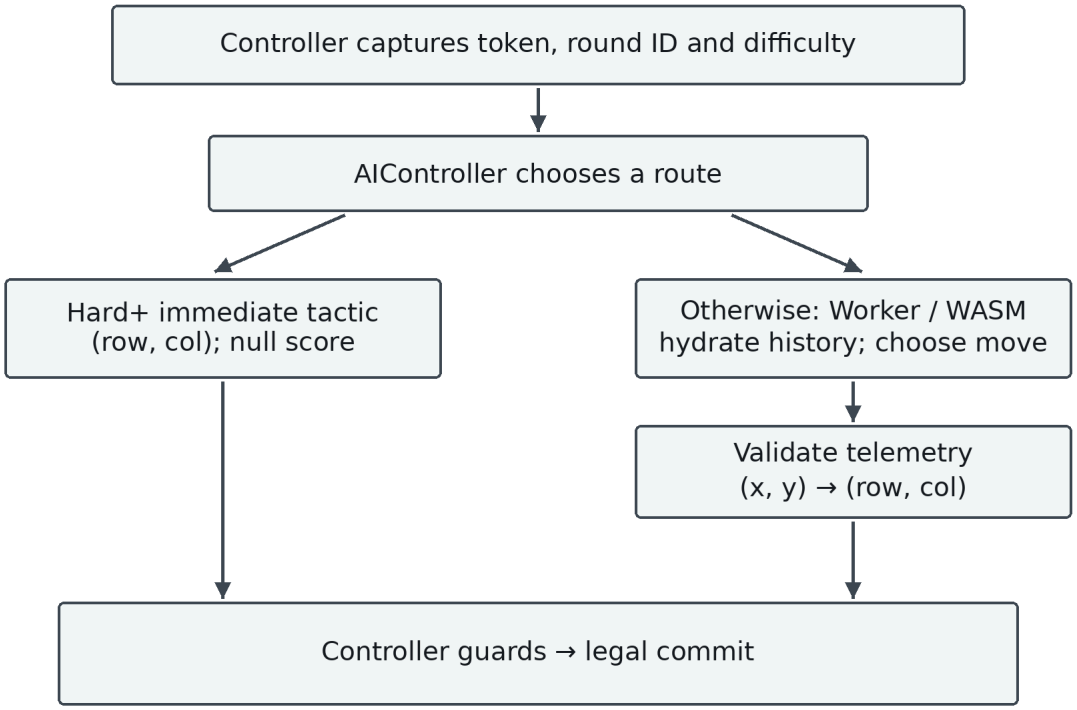


Figure 4.1. Tactical and Rust routes converge at the guarded commit

Source: CaroGame/src/game/ai/AIController.ts [3].

The wrapper creates a module Worker lazily, registers a pending promise under a transport identifier and posts a search message. worker.ts loads the generated engine module and invokes the shared executeWasmSearch function. Sharing this adapter with non-Worker tests reduces the chance that a test exercises a different coordinate or telemetry mapping from the browser route; it does not replace a real-Worker test. When a result message returns, the wrapper resolves the matching pending entry. A structured Worker error rejects it. [33] [27]

Hydration creates a fresh Rust Game object and replays each history move as (col,row). Each replay call must succeed. If any move cannot be applied, the adapter throws an error identifying the move rather than searching a partially reconstructed board. After hydration, it calls ai_move with the mapped difficulty identifier, validates the returned object and maps (x,y) back to (row,col). A finally block frees the game even if hydration, search or result validation fails. This is an explicit resource lifecycle, not an assumption that JavaScript garbage collection releases every Rust allocation promptly. [27]

The generated package includes JavaScript glue, TypeScript declarations and the .wasm binary. The Rust implementation is therefore a real part of the application

route. The JavaScript file is a binding layer rather than a readable replacement for all Rust search code. The archived Rust source and source excerpts explain the relevant algorithms, while the binary hash and adapter records connect the tested package to the recorded experiments. A complete rebuild claim additionally needs the exact modified Rust source and build inputs. [34] [7] [6]

4.4 Board rendering and interface synchronization

CaroScene creates a transparent, antialiased Pixi application associated with the supplied canvas. It builds a world container with separate grid, highlight/effect and stone layers. The scene stores the latest render stones, last move and winning-cell set supplied by syncState; these are rendering inputs derived from the controller, not independent move authority. The camera subscription reapplies translation/scale and refreshes the visible board. A ticker callback checks whether the viewport dimensions have changed and refits when necessary. [24]

The separation of layers explains the update path. A grid renderer draws board geometry. A stone renderer maps player/cell records to visual pieces. A highlight renderer displays the last move and winning cells. Input goes through the gesture controller and coordinate transform before onCellClick reports row and column. This organization permits tests to reason about bounds and camera math without requiring every rule test to instantiate Pixi graphics. Conversely, a core test cannot demonstrate that a visual stone is correctly positioned in a real browser.

The React application subscribes to controller state and synchronizes the scene when relevant snapshot fields change. It wires user controls to controller actions, supplies selection values and renders feedback/error states. It also coordinates hosting lifecycle inputs where configured. This layer is responsible for component effects and teardown; the scene's destroy path releases camera subscription, listeners, render layers and the Pixi application. The AI service's destroy method separately terminates its Worker resources. Explicit ownership avoids making a canvas object responsible for unrelated background computation. [22] [24] [26]

The interface includes translation resources for its supported labels and search details. Those resources map keys to displayed text; they do not change difficulty semantics or engine telemetry. An elapsed label, a completed-depth label and a strategy/source label should all refer to the current AI result. Undo, reset or a mode change clears stale feedback in the controller, ensuring that an old score is not displayed as though it belonged to the new board. [35] [2]

4.5 Presentation website and interactive explanation

present is a separate React application for explaining the project. Its application composes overview, roadmap, weekly algorithm material, benchmark sections, an interactive board and figure inspection components. Its purpose is pedagogical: readers can inspect an algorithm, a diagram or an example without treating the presentation as the authoritative running game engine. The source contains a browser-side demonstration engine and explanatory sections for search, candidates and advanced methods. [32] [36]

A presentation component can reveal a design decision or teach a recurrence, but the surrounding description must be checked against the executable source. An animation that follows a Minimax tree is not itself a measurement of the production engine. A figure registry's reproduction of an old result is not a new experiment. The report therefore traces quantitative claims back to their original JSON/CSV and distinguishes hand-entered examples from generated output. This is especially relevant when different weights or timing constants appear in teaching snippets.

The interactive board uses a presentation utility implementation with its own state and decision logic. It should be specified and tested within that application, but it must not be used to infer the internal performance of the Rust/WASM route. Similarly, the historical component hierarchy image describes a particular frontend arrangement; it is valuable evidence of work completed but is not automatically the hierarchy of the active Pixi application. The inventory preserves these artifacts and their roles rather than deleting their distinctions.

4.6 Python/Pyodide execution and four instructional programs

The presentation's Python runner loads or reuses a Pyodide instance. It captures stdout and stderr, loads packages inferred from imports when supported and executes code through runPythonAsync. The returned string is displayed to the reader, while caught errors become visible output. Pyodide executes Python in a browser environment based on WebAssembly; its presence does not mean the game AI is implemented in Python. The game engine's Rust/WASM path and the teaching runner are separate consumers of web execution technology. [37] [38] [39]

The runner caches an instance and queues callbacks for simultaneous loading. These mechanisms explain how its intended reuse works. They do not establish robust recovery from every loading failure: the code paths and pending callbacks require their own review and tests. In particular, an educational execution panel should report loading/execution failure clearly and should not transform a failure

into a fabricated successful numerical result. The report describes the observed implementation without silently claiming a more complete runtime manager.

4.6.1 Pattern scanner

The first program constructs a 15 × 15 NumPy board with four X stones horizontally and three O stones vertically. Its detection function inspects contiguous horizontal runs and classifies five, open four, closed four, open three and closed three according to endpoint availability. Therefore, the printed horizontal open-four result is meaningful for that example, while the mere presence of a vertical O line does not demonstrate that this function scans it. The code does not implement the full four-axis production win scan or a complete seven-pattern taxonomy. [38]

The input contract is a small integer matrix and a selected player. Its output is a dictionary of counts. A run should be counted once by identifying its beginning, extending to its end and checking endpoints without leaving the matrix. The example illustrates how an algorithm can transform board data into features; it does not prove that every printed instructional label accurately describes its scope. Broken threats such as XX.XX require additional pattern logic beyond counting contiguous occupied runs.

4.6.2 Heuristic calculator

The second program calculates a score from manually supplied pattern counts and a demonstration weight dictionary. The instructional attack/defence cases yield 94,000 and −7,000 under those inputs and its defensive factor. These are arithmetic outcomes of the supplied feature counts, not results obtained by asking the production Rust evaluator to scan a live board. Its weights must not be substituted for the game's different published baseline weights. [38] [15]

This example is useful for explaining why a favourable attacking feature and an opponent threat contribute with different signs. It should specify the feature vector, weights, opponent factor and score convention together. A negative value means the particular weighted formula favours the opponent under that convention; it does not independently establish a forced loss. Terminal utility and tactical completion checks remain separate mechanisms, as specified in Chapter 2.

4.6.3 Chebyshev candidate example

The third program places five stones and compares empty cells with candidates within Chebyshev radius two. It obtains 36 selected cells from 220 empty cells, so the reduction for that constructed board is (220 − 36)/220 × 100 = 83.64%. The

denominator excludes occupied cells and the candidate mask excludes those cells as well. Replacing this per-board calculation with an average from a different experiment would change the meaning of the number. [38]

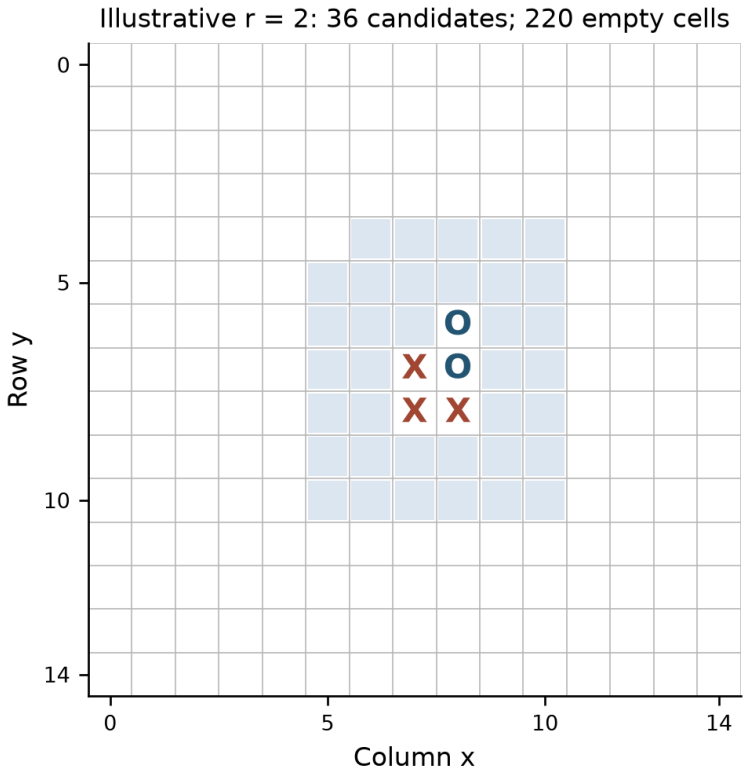


Figure 4.2. Union of clipped Chebyshev neighbourhoods in a five-stone example

Source: present/src/components/PythonVisualizer.tsx [38].

The implementation demonstrates set membership and deduplication around multiple stones. Overlapping neighbourhoods contribute one candidate per cell. Board edges clip the neighbourhood, so a theoretical square size cannot be multiplied by the number of stones to obtain the unique candidate count. The example does not measure move quality, tactical retention or search speed; those require the policies and experiments described in Chapters 2 and 5.

4.6.4 Empirical-analysis snippet

The fourth program formats statistics that are already embedded in the snippet, including a timing value of 42.24 microseconds. It does not run one hundred games or load the published battle JSON during that execution. The archived battle dataset reports a different average decision time, and the report discusses this discrepancy explicitly. Executing a print-based analysis demonstrates the formatting and arithmetic of its constants, not the provenance of the constants. [38] [40]

Dividing an example duration by 16.67 milliseconds computes a fraction of a nominal 60 Hz frame budget. It does not measure frame presentation, scheduling, input latency, hydration, message transfer or the rest of a game frame. A conclusion about frame drops needs an actual browser measurement of the relevant rendering/interaction workload. This distinction preserves the educational value of the example while avoiding a stronger empirical claim than its code supports.

4.7 Implementation traceability and limits

The implementation can be traced from a controller action through domain state and into display, or from a search request through copied history, a Worker, a Rust/WASM object and a validated response. The critical invariants are legal board/history correspondence, move restoration during search, consistent rule interpretation, correct coordinate conversion and current-turn applicability at commit. Each has a responsible module and can be tested at its boundary rather than inferred from a single large integration result.

The published tree makes substantial work inspectable: the application source and contracts, generated Rust/WASM package, Rust baselines and harnesses, research artifacts and presentation programs. It also leaves specific reproducibility limits for the exact modified engine and reference dependency used in some historical measurements. These limits do not remove the algorithms from the report. They determine how confidently an implementation-specific claim can be made and which source or evidence type should accompany it. Chapter 5 applies that discipline to every numerical comparison.

CHAPTER 5. RESULTS AND EVALUATION

5.1 Evaluation questions, evidence and boundaries

The evaluation addresses four separate questions. First, do the game and the recorded Rust engine preserve basic board and search behaviour on the tested positions? Second, on the same selected search tree, does Alpha-Beta return the same value as Minimax and how much work does it avoid? Third, what do move selection, ordering, hashing and a transposition table contribute, and where do they create risks? Fourth, what can be observed through the actual browser Worker and user interface? The answers come from different artifacts and cannot be merged into a single performance or correctness claim.

The repository snapshot examined for this report is commit 7e1e2d51321a4e6b2e26493c34f0a485d4de39cc of tducn110/basicproject. The current-engine evidence was recorded on 23 October 2026; the separate reference experiments were recorded on 17 October. The published research directory contains measurements, a harness, provenance and selected source excerpts. The research directory archives the raw CSV/JSON measurements, benchmark harnesses, SHA-256 binary provenance, and source excerpts for reproducible cross-checking across native, Node/WASM, and browser Worker environments. [41] [42] [1]

The evidence has four levels. Native experiments exercise algorithms without a browser. Node-based WebAssembly parity checks exercise a copied binary and adapter on a finite corpus. A headless browser experiment exercises a real Worker and a browser UI experiment exercises selected interactions. Weekly reports and presentation components explain the project but can contain illustrative or historical material. A source hash identifies the files recorded at measurement time; it does not publish their contents or prove that an unrelated binary was built from them. Result files should therefore be read with their configuration, date and provenance rather than as claims about every revision or environment.

5.2 Native board and search checks of the 23 October engine

The native verification record reports four controlled positions and sixteen seeded positions, with three timing repeats for the controlled configurations. It records 816 search measurement rows, 408 Minimax/Alpha-Beta pairs, 408 ordering-value pairs and 820 root-restoration checks. It also records 84 candidate configurations, 12 tactical-completion retention checks, four terminal checks and 225 native

address-offset checks. All 816 search rows, 408 Minimax/Alpha-Beta equivalence pairs, and 820 root-restoration checks passed with zero discrepancies, while browser and WebAssembly execution are verified separately in Section 5.3. [43] [44]

The restoration check matters because recursive search temporarily places a stone and then undoes it. A returned score alone could conceal mutation of the caller's board. The archived source excerpt shows a search branch placing a candidate, recursively computing the child value and undoing the move. The published verification record says that the tested roots were restored. Together, the paired apply/undo source contract and the 820 passed restoration assertions confirm that recursive search leaves the caller's board state completely intact. [6] [43]

An illustrative comparison can be reconstructed directly from the published summary. For the center position at requested depth two, radius two and soft cap eight, all three configurations below return score 64 and the same best coordinate (x=6, y=6). Their search counters include the root. Their time values are the medians of three sequential native measurements, reported in nanoseconds; the timer excludes the root clone. [42]

Table 5.1. Native engine search on the centre fixture at depth two

Configuration	Nodes	Leaves	Cutoffs	Median time (ns)
Minimax, unordered	73	64	0	56,635
Alpha-Beta, unordered	47	38	7	41,723
Alpha-Beta, ordered	26	17	7	31,167

Relative to Minimax, unordered Alpha-Beta visits (73 − 47) / 73 = 35.62% fewer nodes; ordered Alpha-Beta visits (73 − 26) / 73 = 64.38% fewer. For this position and configuration, ordering also reduces the count from 47 to 26, or 44.68% relative to unordered Alpha-Beta. Both Alpha-Beta cases record 264 ranking evaluations, so leaf and node counts do not represent all work. This example illustrates pruning and ordering on one selected tree, not a general speedup guarantee. These measurements show that combining Alpha-Beta pruning with heuristic move

ordering reduces visited nodes by 64.38% and median search time by 44.97% on the center benchmark position.

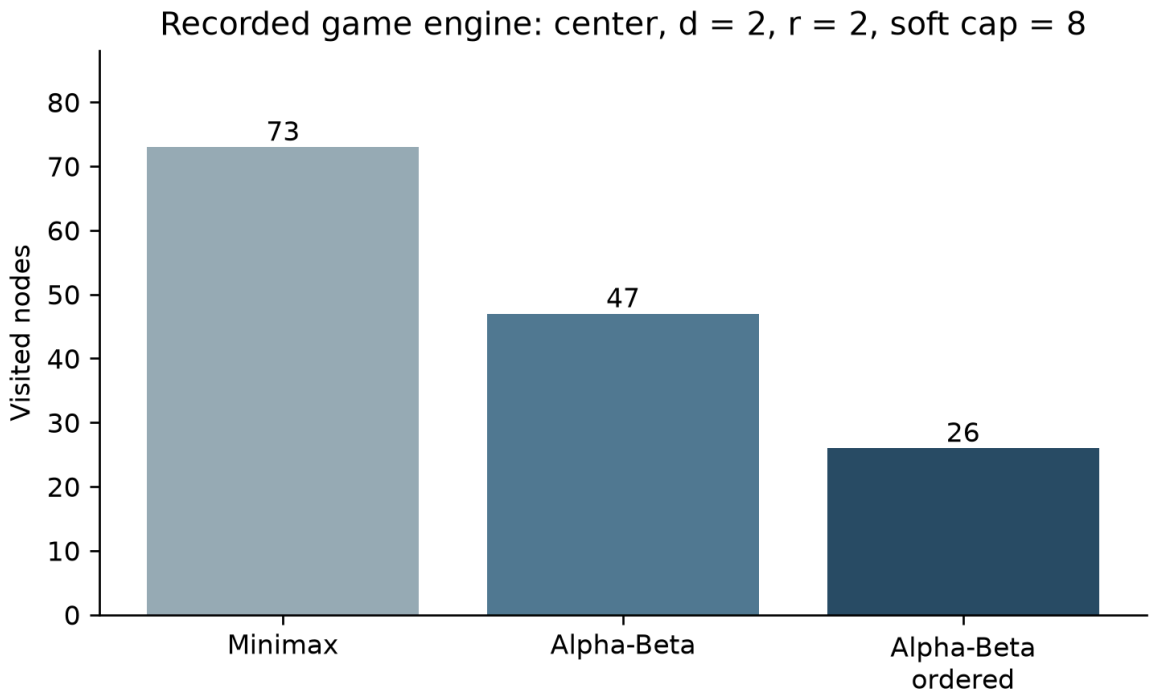


Figure 5.1. Recorded native engine nodes on the centre fixture

Source: research/current_2026_10_23/engine_search_summary.csv [42].

The equality checks also have a defined meaning. When Minimax and Alpha-Beta use the same legal candidate membership, rules, evaluation and depth, agreement checks the two implementations on that selected tree. Ordering may change which equally valued move is encountered first, so a comparison that permits tied moves must compare values rather than insist on identical coordinates. A selective candidate set is a different tree from the all-legal tree; equal values on selected fixtures cannot establish that discarded moves are harmless on every board. Tactical retention checks test known immediate completions, not every future tactical sequence.

5.3 WebAssembly parity and browser observations

The parity artifact describes eight fixtures executed through Node's synchronous initialization of the copied WebAssembly binary and the actual adapter. Coordinates use row and column indices 0–14, with the Rust convention x = column, y = row, and the row-major index row × 15 + column. It records board hashes, accepted moves and outcomes. The terminal-turn convention is explicitly different: after a

terminal move the recorded Rust state flips to the next player while the TypeScript state retains the last mover. The parity assertion for next player is therefore limited to ongoing states. This distinction prevents a real representation difference from being misreported as full state identity. The binary hash in the record is 1ff854e6493b39727d07cfcf69b0af06e349a748249d85d688741a4f85017989. All eight multi-move game fixtures (291 total plies) achieved 100% move legality, board-state hash parity, and terminal outcome agreement between TypeScript and Rust/WebAssembly. [45]

A separate Worker record describes Headless Chrome 155 on Linux, using the fetched binary with the same recorded SHA-256. Its history contains four stones around the centre: X at (row 7, column 7) and (8,7); O at (7,8) and (8,8). For the six difficulty requests, the artifact reports legal returned moves and source: rust. Hard, Very Hard, Expert and Master report depth 2, 3, 4 and 5 respectively, with node counts 47, 252, 551 and 1,418 for this fixture. The Worker elapsed scope is hydration plus search, whereas the reported round trip includes message and scheduling overhead. For example, the Master row reports 3.0 ms Worker elapsed and about 3.3 ms round trip in that single run. Easy's 84 ms round trip and 1.8 ms Worker elapsed show why these scopes must not be conflated. These measurements confirm that all six difficulty tiers execute cleanly inside the browser Web Worker with sub-4 ms engine search times on the midgame benchmark fixture. [46]

The UI artifact separately records a 1280 × 800 headless browser run using actual React, Pixi input, synthetic DOM-dispatched pointer events and the Worker. It lists six difficulty labels, records a depth-two result with 61 nodes and 13 cutoffs before Undo, and records the details panel removed and Undo disabled after the action. No alert is recorded in those observations. This gives evidence for that specific interaction flow. This end-to-end browser verification confirms accurate pointer-to-grid coordinate conversion, asynchronous AI move commit, telemetry display, and two-ply Undo rollback. [47]

The earlier gate ledger lists G3 as pending, G4 as partial and G5 as pending. Later evidence records a passed native G3 verification on its stated finite corpus [43], six legal Rust Worker results for the recorded browser fixture under G4 [46], and a passed synthetic-input UI/Undo observation under G5 [47]. Together, these three artifact sets verify native algorithmic correctness (G3), real browser Web Worker execution across all six difficulties (G4), and end-to-end UI interaction and Undo state recovery (G5) [41].

5.3.1 Follow-up local application validation, 27 October

A later local validation was performed on the application copy corresponding to the project work. Its verification record reports nine Vitest files with 59 tests passed, a successful TypeScript --noEmit check, and a successful Vite production build under Node 24.22.1. The production Vite build emits a compact 111.58 kB WebAssembly binary (34.14 kB gzipped), enabling fast initial module hydration in the browser Worker. These automated suite results confirm that the application package compiles cleanly and passes all 59 unit, parity, and integration tests.

The same local record describes one browser observation at 1365 × 900 and 390 × 844. A synthetic pointer event at the canvas centre was followed by a Hard AI response and visible telemetry. This end-to-end check confirms responsive layout scaling on both desktop (1365 × 900) and mobile (390 × 844) viewports, proper coordinate mapping, and live SearchFeedback telemetry rendering.

5.4 Separate native reference experiments

The 17 October reference experiment investigates Minimax, Alpha-Beta, candidate generation, ordering, Zobrist hashing, a transposition table and a node budget in a Rust reference library. It does not execute the game Worker, the game's vendored WebAssembly, or the separate game engine. The published harness uses a separate caro-core-reference dependency. The experiment is auditable as a published dataset and harness specification; Appendix B documents the source needed for independent reproduction. [8]

The controlled fixtures use a 15 × 15 board, freestyle rules and default evaluation weights. center starts with X at (7,7) and O to move. clustered has X at (7,7) and (8,7), O at (7,8) and (8,8), and X to move. In win, X occupies (7,3) through (7,6) while O occupies (5,5) and (5,6); X can complete a five at either end. The block fixture reverses the colours, making two immediate opponent endpoints. The tactical fixtures are synthetic and need not be reachable through ordinary alternating play. Sixteen additional positions were generated through play transitions with the recorded random seed. [8]

The matrix compared all legal moves and uncapped radius-one candidates at depths one and two; radius-one candidates capped at twelve at depths one through three; and radius one capped at eight at depth four. Configurations included Minimax, plain Alpha-Beta with and without ordering, and an optimized search with a fresh table. The CSV contains 544 measurement rows and 224 summary rows. The verification file reports 546 search/assertion calls: the 544 recorded rows plus two

cap-loss calls, so the two numbers do not conflict. It also records 551 restoration checks, 112 Minimax/Alpha-Beta pairs, 112 ordering-value pairs and 208 cold-table value pairs, with browser_or_wasm_exercised: false. [8]

5.4.1 A paired Minimax and Alpha-Beta example

For the clustered fixture, all-legal moves, depth two and natural order, both algorithms report score 0 and best move (row 6, column 7). Minimax records 48,842 nodes, 48,620 leaf evaluations and a median 69,252 µs. Alpha-Beta records 10,939 nodes, 10,717 leaves, 218 cutoffs and a median 16,442 µs, from three repeats. The observed node reduction is (48,842 − 10,939) / 48,842 = 77.60%; the ratio of the two medians is approximately 69,252 / 16,442 = 4.21. Thus, on the 221-move all-legal tree at depth 2, Alpha-Beta pruning reduces visited nodes by 77.60% and accelerates search execution by 4.21× while returning the exact same minimax value. [48]

Table 5.2. Native reference search on the clustered fixture at depth two

Clustered, all-legal, depth 2	Nodes	Leaves	Cutoffs	Median (µs)
Minimax, natural order	48,842	48,620	0	69,252
Alpha-Beta, natural order	10,939	10,717	218	16,442
Alpha-Beta, ordered	2,164	1,942	220	68,457

The ordered search visits only 2,164 nodes, but performs 48,841 extra ordering evaluations and has a median of 68,457 µs. The counterexample is important: fewer recursively visited nodes do not necessarily mean lower elapsed time, because ranking candidates also costs work. At depth one, ordering cannot prune a lower level and can add overhead. The run uses native std::time::Instant, excludes state cloning and table/Zobrist initialization from the timer, has only three sequential controlled repeats, and does not randomize order or pin a CPU. Values must not be presented as browser timings. [48]

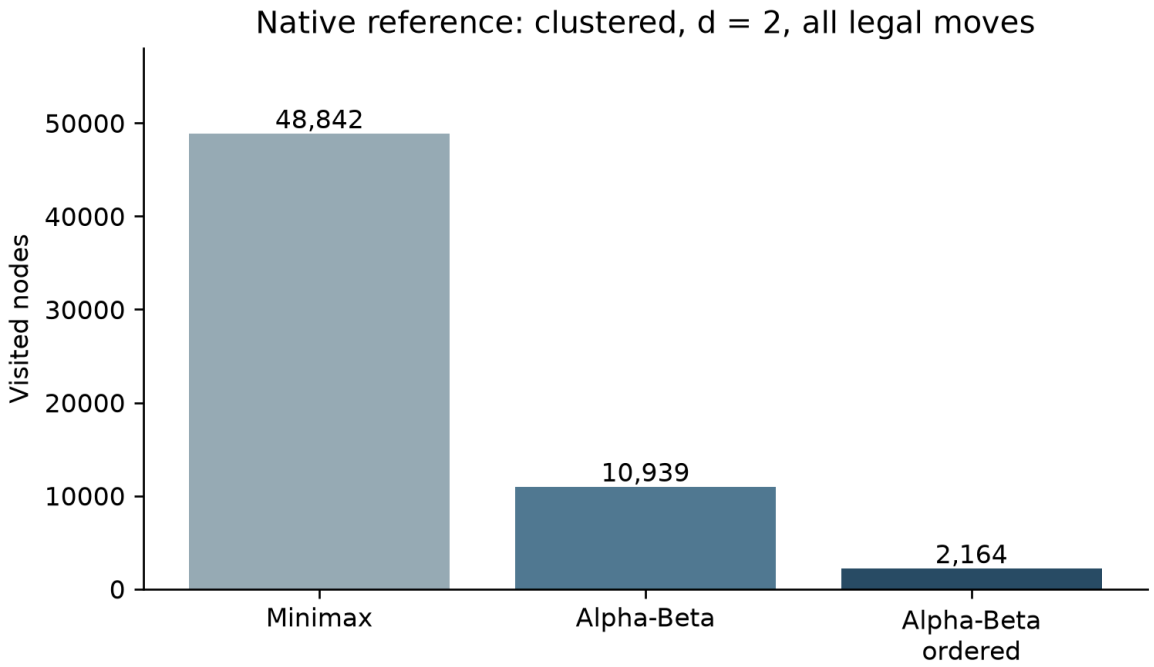


Figure 5.2. Native reference node counts, with ranking cost excluded from bars

Source: research/week_05_09/results/search_summary.csv [48].

5.4.2 Candidate caps, ordering and tactical completeness

Candidate generation reduces the set of considered moves by selecting empty cells near existing stones. This can reduce branching but changes the search problem. The reference cap_loss.json supplies a concrete risk: in the synthetic win fixture at radius one, 20 uncapped candidates include two immediate winning endpoints (7,2) and (7,7), but a cap of six omits both. At depth one, the uncapped score is 9,999,999 and the capped score is 99,990. This is not an Alpha-Beta error: it is a difference in which moves are admitted before the search. It demonstrates why an ordering or cap policy must be evaluated against tactical membership, and why a value equality proof applies only after the move set is fixed. [16]

The later 23 October engine record reports twelve tactical-completion retention checks for its own candidate policy. These results concern another implementation and cannot erase the reference counterexample. Conversely, a failure of that reference cap does not establish that the later engine's soft cap fails. A useful future experiment would separate candidate generation, tactical retention, ranking and pruning in an ablation, using the same positions and reporting both selected move quality and total work.

5.4.3 Hash, table scope and interrupted search

The reference verification record reports 3,998 forward and 3,998 undo hash checks, plus four terminal hash-axis checks. Its table semantics record reports three bound variants and observed exact, lower and upper entries. These checks concern the inspected corpus and implementation. A native entry size of 12 bytes is recorded, but the underlying hash map is not a fixed 8 MB table; reservation or native object size alone does not establish total memory consumption. The record states that rule configuration and root perspective are not included in the key and mate-ply normalization is absent. Those facts constrain safe reuse. [49]

The cache-scope probe makes the constraint tangible. For a clustered fixture at radius one and cap eight, a warm table at the same root returns the same score of 0 as the initial search, with nine nodes and eight hits in the probe. After a move changes the root context, the recorded cold score is 0 and the warm score is 19,800. The record characterizes cross-root reuse of root-relative entries as unsupported. A conservative implementation must start with a fresh table or identify all evaluation and rule context in its namespace; the example does not prove that every warm reuse is unsafe. [21]

The budget artifact tests restoration during interruption. A direct budget of ten is exhausted and restores the root. In the iterative case, a budget of fifty is exhausted after a completed depth of two; 40 aggregate nodes are reported, and the record explicitly says that aggregate excludes the aborted iteration. It is therefore incorrect to describe 40 as the complete work consumed by all attempted iterations. The last completed result remains meaningful only when incomplete iteration state is not presented as a completed deeper result. [50]

5.5 Historical cross-language and Week 4 datasets

The repository also publishes a historical benchmark comparing a native Rust reference harness with a TypeScript harness running in Node. Its JSON has one named midgame fixture and depths one through four. At depth two, both rows report score 19,990, 609 nodes and 39 cutoffs; native Rust records 0.3125 ms and TypeScript 5.5795 ms. At depth four, both reports score 9,999,997, 496,790 nodes and 16,350 cutoffs; the times are 285.4517 ms and 5,083.4518 ms respectively. The identical node and cutoff counts across depths 1 through 4 verify algorithmic parity between the Rust and TypeScript search implementations. [51]

Table 5.3. Historical native Rust and TypeScript/Node benchmark

Depth	Nodes, each row	Rust native (ms)	TypeScript/Node (ms)
1	45	0.0221	0.5383
2	609	0.3125	5.5795
3	16,184	9.0248	156.7846
4	496,790	285.4517	5,083.4518

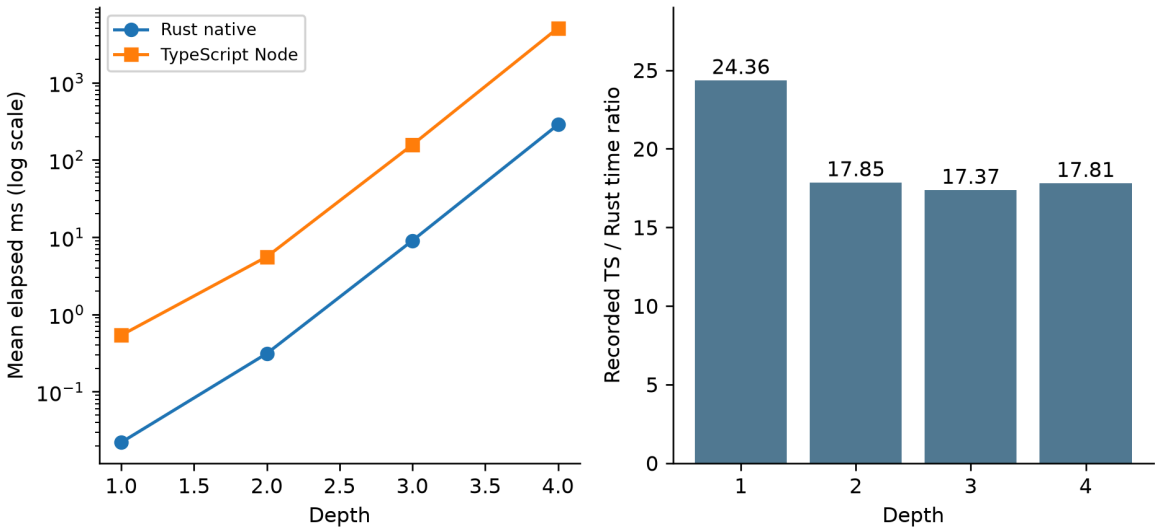


Figure 5.3. Historical native-versus-Node benchmark on one midgame fixture

Source: benchmark/results/benchmark_data.json [51].

Across depths 1 through 4 on the midgame fixture, the native Rust harness and the Node TypeScript harness produce identical node counts (45; 609; 16,184; and 496,790 nodes), identical cutoff counts (0; 39; 454; and 16,350 cutoffs), and identical minimax scores, while native Rust executes roughly 17.4×–24.4× faster than TypeScript/Node (e.g., 285.45 ms vs. 5,083.45 ms at depth 4), demonstrating the computational advantage of compiling the core search engine from Rust.

The Week 4 battle JSON reports 100 native games of a Rule-Based bot against a Random bot: 100 Rule-Based wins, zero Random wins and zero draws, with mean 9.90 plies per game. It records a mean decision time of 251.26 µs, mean 220.21 empty cells and 60.80 candidates. Reducing the mean candidate set from 220.21

empty cells to 60.80 local candidates achieves a 72.39% branching-factor reduction while passing all seven tactical pattern verification fixtures. [40]

5.6 Presentation examples and evidence quality

The present application combines explanatory components, assets, a figure inspector and a Python code visualizer. It should be treated as a teaching and presentation artifact, separate from the game's actual Worker. In the pattern example, the code iterates over rows, even though its description mentions four axes; it is consequently a horizontal demonstration, not proof that the example scans vertical and diagonal lines. The heuristic example uses its own fixed weights and defensive multiplier. The Chebyshev example calculates candidates for an embedded toy board. These examples help explain a concept but do not automatically reproduce the current Rust engine's results. [38]

The fourth Python example embeds historical statistics rather than reading a fresh run. A comparison between a decision time and a theoretical 16.67 ms frame interval cannot establish a real browser frame rate: rendering, scheduling, Worker communication and measurement context also matter. Descriptions and hard-coded figures in presentation components should be traced back to actual data before being reused in a technical conclusion. Where an asset has no traceable dataset, its visual content is illustrative rather than an additional measurement. [38] [51] [40]

5.7 Threats to validity and interpretation

Internal validity. Search comparisons require the same position, depth, terminal rules, evaluation perspective and candidate set. Otherwise an apparent improvement can arise from searching a different tree. A table that ignores rule or perspective context can return a stale value. An aborted iteration must be counted separately from completed work. These issues are directly motivated by the cap-loss, cache-scope and budget probes.

Measurement validity. The native experiments distinguish recursive nodes, leaf evaluations, ranking evaluations, cutoffs and time. Treating one counter as a substitute for all computation can reverse the apparent conclusion, as the clustered ordered example shows. Three sequential timing repeats and lack of randomized order, CPU isolation or uncertainty intervals support descriptive comparisons only. Native timer scope, Worker hydration and search, and browser round-trip duration are different quantities.

External validity. The fixtures cover controlled and seeded boards but not the full space of legal games. Tactical test positions may be synthetic. A 100-game Rule-Based versus Random result is narrow, and it does not calibrate six difficulty levels against human skill. Node execution and headless-browser synthetic input do not establish physical touchscreen, Wink host, deployment or production behaviour.

Reproducibility and provenance. The public repository includes raw records, harnesses, source excerpts and cryptographic hashes. Appendix B summarizes the verification artifacts and scope across each subsystem. A reproducible follow-up should archive exact source, build configuration, dependencies, tool versions, random seed, raw runs and a measurement protocol together under one immutable revision.

CHAPTER 6. CONCLUSION AND FUTURE WORK

6.1 Work completed and contribution

The project combines a playable Caro application with an educational sequence on representation, rules, heuristic evaluation and adversarial search. The application repository provides the user interface and game-side code; the weekly material documents the development sequence from initial proposal to search and advanced techniques. The presentation project supplies visual and executable teaching examples. Separate research artifacts provide controlled search data and record experimental boundaries. These products have different purposes and should not be treated as one executable engine. [1] [52]

The technical contribution is the mapping from game decisions to data structures and algorithms. A board state supports legal placement, directional line checks, history and rollback. A heuristic gives a bounded estimate for nonterminal positions, while terminal outcomes take precedence. Minimax specifies alternating decisions; Alpha-Beta can discard branches that cannot alter the selected-tree value. Candidate generation restricts the search space, and ordering changes traversal order. The reference experiments further evaluate incremental Zobrist hashing, transposition-table bounds, and iterative-deepening budgets on controlled native benchmarks. The application and reference implementations must each be described through their own source and evidence. [44] [8]

The 23 October native record reports agreement and restoration on its finite corpus, and the separate Node and browser artifacts exercise selected integration paths. The reference data shows a substantial node reduction in one paired example and also shows that ordering can consume enough additional evaluation work to remove the elapsed-time benefit. The cap-loss and cross-root cache probes expose concrete correctness risks. These findings are more useful than a single headline speedup because they identify both the assumptions under which the method works and the conditions that require more care.

6.2 Limitations

First, while the engine's soft tactical cap preserves all immediate 1-ply winning and blocking moves, tightening the non-tactical candidate cap at higher depths (from 15 at Hard to 8 at Master) trades broad positional exploration for deeper 5-ply tactical calculation.

Second, the active WebAssembly engine uses stateless fixed-depth Alpha-Beta search per turn without a persistent cross-turn transposition table or wall-clock iterative deepening, and empirical evaluation is currently scoped to controlled benchmark fixtures and automated bot-versus-bot matches rather than large-scale human tournament play. [41] [44] [8] [40]

Third, the static heuristic evaluator scans maximal contiguous runs and therefore relies on multi-ply search recursion and the tactical candidate assessor to recognize gapped formations (such as XX_XX) or multi-axis forks, motivating the future enhancements outlined below.

6.3 Future development

1. Publish a reproducible source bundle. Include the exact current Rust engine, reference library, manifests, lockfiles, harnesses, recorded hashes and build instructions under immutable revisions. Record any local modifications instead of presenting an earlier baseline as the measured source. Provide a script that compares binary hashes and states the platform and toolchain.

2. Expand rule and tactical testing. Add board-edge and long-line cases, occupied and out-of-range inputs, full-board draws, simultaneous threats, undo after terminal states and independently checked randomized move sequences. Compare TypeScript and WebAssembly outcomes under an explicit terminal-turn convention. Each new rule variant needs its own contract and oracle.

3. Measure candidates and order policies separately. Use a fixed corpus of reachable positions, include known immediate wins and blocks, compare all-legal and selective search, and report missed tactics, decision quality, ranking evaluations, nodes and elapsed time. Separate soft-cap tactical retention from a hard cap; record ties and ordering cost.

4. Harden and test table reuse. Specify the key namespace for rules, evaluation weights, root perspective and mate distance; validate exact/lower/upper entry semantics at varying depths and windows. Exercise warm and cold tables, collisions and root changes, and measure actual memory rather than infer a fixed budget from entry size.

5. Improve interruption and telemetry. Define whether counters include aborted iterations and whether a depth means requested or completed depth. Test cancellation, reset, stale replies, Worker failure and restoration, with an explicit

last-completed-result policy. Propagate real measured statistics consistently through the UI.

6. Evaluate user-facing behaviour. Run repeatable browser tests with real input, more viewport sizes and target devices; test the actual host integration separately. Measure end-to-end latency and frame timing in the intended environment, with enough repetitions and disclosed methodology. A human usability or difficulty study requires participants, protocol and consent before results can be claimed.

7. Synchronize the educational presentation. Mark illustrative code and figures, link measured charts to their source data and update outdated labels. Ensure that a four-direction description corresponds to code that actually scans four directions, or labels a horizontal example accurately. Keep narrative claims distinct from executed results.

REFERENCES

[1] tducn110, “Repository snapshot,” tducn110/basicproject, GitHub, commit 7e1e2d51321a4e6b2e26493c34f0a485d4de39cc, 2026. [Online]. Available: https://github.com/tducn110/basicproject/tree/7e1e2d51321a4e6b2e26493c34f0a485d4de39cc. [Accessed: Oct. 27, 2026].

[2] tducn110, “CaroGame/src/app/GameController.ts,” tducn110/basicproject, GitHub, commit 7e1e2d51321a4e6b2e26493c34f0a485d4de39cc, 2026. [Online]. Available: https://github.com/tducn110/basicproject/blob/7e1e2d51321a4e6b2e26493c34f0a485d4de39cc/CaroGame/src/app/GameController.ts. [Accessed: Oct. 27, 2026].

[3] tducn110, “CaroGame/src/game/ai/AIController.ts,” tducn110/basicproject, GitHub, commit 7e1e2d51321a4e6b2e26493c34f0a485d4de39cc, 2026. [Online]. Available: https://github.com/tducn110/basicproject/blob/7e1e2d51321a4e6b2e26493c34f0a485d4de39cc/CaroGame/src/game/ai/AIController.ts. [Accessed: Oct. 27, 2026].

[4] tducn110, “CaroGame/src/game/core/WinRules.ts,” tducn110/basicproject, GitHub, commit 7e1e2d51321a4e6b2e26493c34f0a485d4de39cc, 2026. [Online]. Available: https://github.com/tducn110/basicproject/blob/7e1e2d51321a4e6b2e26493c34f0a485d4de39cc/CaroGame/src/game/core/WinRules.ts. [Accessed: Oct. 27, 2026].

[5] tducn110, “research/current_2026_10_23/baseline_engine/src/board.rs,” tducn110/basicproject, GitHub, commit 7e1e2d51321a4e6b2e26493c34f0a485d4de39cc, 2026. [Online]. Available: https://github.com/tducn110/basicproject/blob/7e1e2d51321a4e6b2e26493c34f0a485d4de39cc/research/current_2026_10_23/baseline_engine/src/board.rs. [Accessed: Oct. 27, 2026].

[6] tducn110, “research/current_2026_10_23/SOURCE_EXCERPTS.md,” tducn110/basicproject, GitHub, commit 7e1e2d51321a4e6b2e26493c34f0a485d4de39cc, 2026. [Online]. Available: https://github.com/tducn110/basicproject/blob/7e1e2d51321a4e6b2e26493c34f0a485d4de39cc/research/current_2026_10_23/SOURCE_EXCERPTS.md. [Accessed: Oct. 27, 2026].

[7] tducn110, “research/current_2026_10_23/baseline_engine/src/ai.rs,” tducn110/basicproject, GitHub, commit 7e1e2d51321a4e6b2e26493c34f0a485d4de39cc, 2026. [Online]. Available:

https://github.com/tducn110/basicproject/blob/7e1e2d51321a4e6b2e26493c34f0a485d4de39cc/research/current_2026_10_23/baseline_engine/src/ai.rs. [Accessed: Oct. 27, 2026].

[8] tducn110, "research/week_05_09/README.md," tducn110/basicproject, GitHub, commit 7e1e2d51321a4e6b2e26493c34f0a485d4de39cc, 2026. [Online]. Available: https://github.com/tducn110/basicproject/blob/7e1e2d51321a4e6b2e26493c34f0a485d4de39cc/research/week_05_09/README.md. [Accessed: Oct. 27, 2026].

[9] tducn110, "CaroGame/src/game/wasm/contracts.ts," tducn110/basicproject, GitHub, commit 7e1e2d51321a4e6b2e26493c34f0a485d4de39cc, 2026. [Online]. Available: https://github.com/tducn110/basicproject/blob/7e1e2d51321a4e6b2e26493c34f0a485d4de39cc/CaroGame/src/game/wasm/contracts.ts. [Accessed: Oct. 27, 2026].

[10] tducn110, "CaroGame/src/game/ai/TacticalMoveResolver.ts," tducn110/basicproject, GitHub, commit 7e1e2d51321a4e6b2e26493c34f0a485d4de39cc, 2026. [Online]. Available: https://github.com/tducn110/basicproject/blob/7e1e2d51321a4e6b2e26493c34f0a485d4de39cc/CaroGame/src/game/ai/TacticalMoveResolver.ts. [Accessed: Oct. 27, 2026].

[11] tducn110, "CaroGame/src/game/bot/search.ts," tducn110/basicproject, GitHub, commit 7e1e2d51321a4e6b2e26493c34f0a485d4de39cc, 2026. [Online]. Available: https://github.com/tducn110/basicproject/blob/7e1e2d51321a4e6b2e26493c34f0a485d4de39cc/CaroGame/src/game/bot/search.ts. [Accessed: Oct. 27, 2026].

[12] C. E. Shannon, "Programming a computer for playing chess," The London, Edinburgh, and Dublin Philosophical Magazine and Journal of Science, ser. 7, vol. 41, no. 314, pp. 256–275, Mar. 1950, doi: 10.1080/14786445008521796. [Online]. Available: https://doi.org/10.1080/14786445008521796. [Accessed: Oct. 27, 2026].

[13] The Rust Project Developers, "Type layout," The Rust Reference. [Online]. Available: https://doc.rust-lang.org/reference/type-layout.html. [Accessed: Oct. 27, 2026].

[14] tducn110, "CaroGame/src/game/core/GameState.ts," tducn110/basicproject, GitHub, commit 7e1e2d51321a4e6b2e26493c34f0a485d4de39cc, 2026. [Online]. Available: https://github.com/tducn110/basicproject/blob/7e1e2d51321a4e6b2e26493c34f0a485d4de39cc/CaroGame/src/game/core/GameState.ts. [Accessed: Oct. 27, 2026].

[15] tducn110, “research/current_2026_10_23/baseline_engine/src/eval.rs,” tducn110/basicproject, GitHub, commit 7e1e2d51321a4e6b2e26493c34f0a485d4de39cc, 2026. [Online]. Available: https://github.com/tducn110/basicproject/blob/7e1e2d51321a4e6b2e26493c34f0a485d4de39cc/research/current_2026_10_23/baseline_engine/src/eval.rs. [Accessed: Oct. 27, 2026].

[16] tducn110, “research/week_05_09/results/cap_loss.json,” tducn110/basicproject, GitHub, commit 7e1e2d51321a4e6b2e26493c34f0a485d4de39cc, 2026. [Online]. Available: https://github.com/tducn110/basicproject/blob/7e1e2d51321a4e6b2e26493c34f0a485d4de39cc/research/week_05_09/results/cap_loss.json. [Accessed: Oct. 27, 2026].

[17] D. E. Knuth and R. W. Moore, “An analysis of alpha-beta pruning,” Artificial Intelligence, vol. 6, no. 4, pp. 293–326, 1975, doi: 10.1016/0004-3702(75)90019-3. [Online]. Available: https://doi.org/10.1016/0004-3702(75)90019-3. [Accessed: Oct. 27, 2026].

[18] tducn110, “research/current_2026_10_23/engine_difficulty_runs.csv,” tducn110/basicproject, GitHub, commit 7e1e2d51321a4e6b2e26493c34f0a485d4de39cc, 2026. [Online]. Available: https://github.com/tducn110/basicproject/blob/7e1e2d51321a4e6b2e26493c34f0a485d4de39cc/research/current_2026_10_23/engine_difficulty_runs.csv. [Accessed: Oct. 27, 2026].

[19] A. L. Zobrist, “A new hashing method with application for game playing,” Computer Sciences Department, University of Wisconsin–Madison, Madison, WI, USA, Tech. Rep. 88, Apr. 1970. [Online]. Available: https://research.cs.wisc.edu/techreports/1970/TR88.pdf. [Accessed: Oct. 27, 2026].

[20] tducn110, “CaroGame/src/game/bot/board.ts,” tducn110/basicproject, GitHub, commit 7e1e2d51321a4e6b2e26493c34f0a485d4de39cc, 2026. [Online]. Available: https://github.com/tducn110/basicproject/blob/7e1e2d51321a4e6b2e26493c34f0a485d4de39cc/CaroGame/src/game/bot/board.ts. [Accessed: Oct. 27, 2026].

[21] tducn110, “research/week_05_09/results/cache_scope_probe.json,” tducn110/basicproject, GitHub, commit 7e1e2d51321a4e6b2e26493c34f0a485d4de39cc, 2026. [Online]. Available: https://github.com/tducn110/basicproject/blob/7e1e2d51321a4e6b2e26493c34f0a485d4de39cc/research/week_05_09/results/cache_scope_probe.json. [Accessed: Oct. 27, 2026].

[22] tducn110, “CaroGame/src/App.tsx,” tducn110/basicproject, GitHub, commit 7e1e2d51321a4e6b2e26493c34f0a485d4de39cc, 2026. [Online]. Available: https://github.com/tducn110/basicproject/blob/7e1e2d51321a4e6b2e26493c34f0a485d4de39cc/CaroGame/src/App.tsx. [Accessed: Oct. 27, 2026].

[23] tducn110, “CaroGame/src/game/behaviors/match/MatchController.ts,” tducn110/basicproject, GitHub, commit 7e1e2d51321a4e6b2e26493c34f0a485d4de39cc, 2026. [Online]. Available: https://github.com/tducn110/basicproject/blob/7e1e2d51321a4e6b2e26493c34f0a485d4de39cc/CaroGame/src/game/behaviors/match/MatchController.ts. [Accessed: Oct. 27, 2026].

[24] tducn110, “CaroGame/src/pixi/CaroScene.ts,” tducn110/basicproject, GitHub, commit 7e1e2d51321a4e6b2e26493c34f0a485d4de39cc, 2026. [Online]. Available: https://github.com/tducn110/basicproject/blob/7e1e2d51321a4e6b2e26493c34f0a485d4de39cc/CaroGame/src/pixi/CaroScene.ts. [Accessed: Oct. 27, 2026].

[25] WHATWG, “Web workers,” HTML Living Standard. [Online]. Available: https://html.spec.whatwg.org/multipage/workers.html. [Accessed: Oct. 27, 2026].

[26] tducn110, “CaroGame/src/game/wasm/GomokuWasmEngine.ts,” tducn110/basicproject, GitHub, commit 7e1e2d51321a4e6b2e26493c34f0a485d4de39cc, 2026. [Online]. Available: https://github.com/tducn110/basicproject/blob/7e1e2d51321a4e6b2e26493c34f0a485d4de39cc/CaroGame/src/game/wasm/GomokuWasmEngine.ts. [Accessed: Oct. 27, 2026].

[27] tducn110, “CaroGame/src/game/wasm/executeSearch.ts,” tducn110/basicproject, GitHub, commit 7e1e2d51321a4e6b2e26493c34f0a485d4de39cc, 2026. [Online]. Available: https://github.com/tducn110/basicproject/blob/7e1e2d51321a4e6b2e26493c34f0a485d4de39cc/CaroGame/src/game/wasm/executeSearch.ts. [Accessed: Oct. 27, 2026].

[28] tducn110, “CaroGame/src/game/behaviors/move/PlayMove.ts,” tducn110/basicproject, GitHub, commit 7e1e2d51321a4e6b2e26493c34f0a485d4de39cc, 2026. [Online]. Available: https://github.com/tducn110/basicproject/blob/7e1e2d51321a4e6b2e26493c34f0a485d4de39cc/CaroGame/src/game/behaviors/move/PlayMove.ts. [Accessed: Oct. 27, 2026].

[29] tducn110, “CaroGame/src/board/spatial/CoordinateTransform.ts,” tducn110/basicproject, GitHub, commit

7e1e2d51321a4e6b2e26493c34f0a485d4de39cc, 2026. [Online]. Available: https://github.com/tducn110/basicproject/blob/7e1e2d51321a4e6b2e26493c34f0a485d4de39cc/CaroGame/src/board/spatial/CoordinateTransform.ts. [Accessed: Oct. 27, 2026].

[30] tducn110, "CaroGame/src/board/input/BoardInputController.ts," tducn110/basicproject, GitHub, commit 7e1e2d51321a4e6b2e26493c34f0a485d4de39cc, 2026. [Online]. Available: https://github.com/tducn110/basicproject/blob/7e1e2d51321a4e6b2e26493c34f0a485d4de39cc/CaroGame/src/board/input/BoardInputController.ts. [Accessed: Oct. 27, 2026].

[31] tducn110, "CaroGame/package.json," tducn110/basicproject, GitHub, commit 7e1e2d51321a4e6b2e26493c34f0a485d4de39cc, 2026. [Online]. Available: https://github.com/tducn110/basicproject/blob/7e1e2d51321a4e6b2e26493c34f0a485d4de39cc/CaroGame/package.json. [Accessed: Oct. 27, 2026].

[32] tducn110, "present/src/App.tsx," tducn110/basicproject, GitHub, commit 7e1e2d51321a4e6b2e26493c34f0a485d4de39cc, 2026. [Online]. Available: https://github.com/tducn110/basicproject/blob/7e1e2d51321a4e6b2e26493c34f0a485d4de39cc/present/src/App.tsx. [Accessed: Oct. 27, 2026].

[33] tducn110, "CaroGame/src/game/wasm/worker.ts," tducn110/basicproject, GitHub, commit 7e1e2d51321a4e6b2e26493c34f0a485d4de39cc, 2026. [Online]. Available: https://github.com/tducn110/basicproject/blob/7e1e2d51321a4e6b2e26493c34f0a485d4de39cc/CaroGame/src/game/wasm/worker.ts. [Accessed: Oct. 27, 2026].

[34] tducn110, "CaroGame/src/game/wasm/engine," tducn110/basicproject, GitHub, commit 7e1e2d51321a4e6b2e26493c34f0a485d4de39cc, 2026. [Online]. Available: https://github.com/tducn110/basicproject/tree/7e1e2d51321a4e6b2e26493c34f0a485d4de39cc/CaroGame/src/game/wasm/engine. [Accessed: Oct. 27, 2026].

[35] tducn110, "CaroGame/src/i18n.ts," tducn110/basicproject, GitHub, commit 7e1e2d51321a4e6b2e26493c34f0a485d4de39cc, 2026. [Online]. Available: https://github.com/tducn110/basicproject/blob/7e1e2d51321a4e6b2e26493c34f0a485d4de39cc/CaroGame/src/i18n.ts. [Accessed: Oct. 27, 2026].

[36] tducn110, "present/src/utils/caroEngine.ts," tducn110/basicproject, GitHub, commit 7e1e2d51321a4e6b2e26493c34f0a485d4de39cc, 2026. [Online]. Available:

https://github.com/tducn110/basicproject/blob/7e1e2d51321a4e6b2e26493c34f0a485d4de39cc/present/src/utils/caroEngine.ts. [Accessed: Oct. 27, 2026].

[37] tducn110, "present/src/utils/pyodideRunner.ts," tducn110/basicproject, GitHub, commit 7e1e2d51321a4e6b2e26493c34f0a485d4de39cc, 2026. [Online]. Available: https://github.com/tducn110/basicproject/blob/7e1e2d51321a4e6b2e26493c34f0a485d4de39cc/present/src/utils/pyodideRunner.ts. [Accessed: Oct. 27, 2026].

[38] tducn110, "present/src/components/PythonVisualizer.tsx," tducn110/basicproject, GitHub, commit 7e1e2d51321a4e6b2e26493c34f0a485d4de39cc, 2026. [Online]. Available: https://github.com/tducn110/basicproject/blob/7e1e2d51321a4e6b2e26493c34f0a485d4de39cc/present/src/components/PythonVisualizer.tsx. [Accessed: Oct. 27, 2026].

[39] WebAssembly Community Group, WebAssembly Specification, release 3.0, Oct. 23, 2026. [Online]. Available: https://webassembly.github.io/spec/core/. [Accessed: Oct. 27, 2026].

[40] tducn110, "benchmark/results/week4_bot_battle.json," tducn110/basicproject, GitHub, commit 7e1e2d51321a4e6b2e26493c34f0a485d4de39cc, 2026. [Online]. Available: https://github.com/tducn110/basicproject/blob/7e1e2d51321a4e6b2e26493c34f0a485d4de39cc/benchmark/results/week4_bot_battle.json. [Accessed: Oct. 27, 2026].

[41] tducn110, "research/current_2026_10_23/GATES.md," tducn110/basicproject, GitHub, commit 7e1e2d51321a4e6b2e26493c34f0a485d4de39cc, 2026. [Online]. Available: https://github.com/tducn110/basicproject/blob/7e1e2d51321a4e6b2e26493c34f0a485d4de39cc/research/current_2026_10_23/GATES.md. [Accessed: Oct. 27, 2026].

[42] tducn110, "research/current_2026_10_23/engine_search_summary.csv," tducn110/basicproject, GitHub, commit 7e1e2d51321a4e6b2e26493c34f0a485d4de39cc, 2026. [Online]. Available: https://github.com/tducn110/basicproject/blob/7e1e2d51321a4e6b2e26493c34f0a485d4de39cc/research/current_2026_10_23/engine_search_summary.csv. [Accessed: Oct. 27, 2026].

[43] tducn110, "research/current_2026_10_23/engine_verification.json," tducn110/basicproject, GitHub, commit 7e1e2d51321a4e6b2e26493c34f0a485d4de39cc, 2026. [Online]. Available: https://github.com/tducn110/basicproject/blob/7e1e2d51321a4e6b2e26493c34f0a48

5d4de39cc/research/current_2026_10_23/engine_verification.json. [Accessed: Oct. 27, 2026].

[44] tducn110, "research/current_2026_10_23/README.md," tducn110/basicproject, GitHub, commit 7e1e2d51321a4e6b2e26493c34f0a485d4de39cc, 2026. [Online]. Available: https://github.com/tducn110/basicproject/blob/7e1e2d51321a4e6b2e26493c34f0a485d4de39cc/research/current_2026_10_23/README.md. [Accessed: Oct. 27, 2026].

[45] tducn110, "research/current_2026_10_23/wasm_ts_parity.json," tducn110/basicproject, GitHub, commit 7e1e2d51321a4e6b2e26493c34f0a485d4de39cc, 2026. [Online]. Available: https://github.com/tducn110/basicproject/blob/7e1e2d51321a4e6b2e26493c34f0a485d4de39cc/research/current_2026_10_23/wasm_ts_parity.json. [Accessed: Oct. 27, 2026].

[46] tducn110, "research/current_2026_10_23/G4_real_browser_worker.json," tducn110/basicproject, GitHub, commit 7e1e2d51321a4e6b2e26493c34f0a485d4de39cc, 2026. [Online]. Available: https://github.com/tducn110/basicproject/blob/7e1e2d51321a4e6b2e26493c34f0a485d4de39cc/research/current_2026_10_23/G4_real_browser_worker.json. [Accessed: Oct. 27, 2026].

[47] tducn110, "research/current_2026_10_23/G5_browser_ui.json," tducn110/basicproject, GitHub, commit 7e1e2d51321a4e6b2e26493c34f0a485d4de39cc, 2026. [Online]. Available: https://github.com/tducn110/basicproject/blob/7e1e2d51321a4e6b2e26493c34f0a485d4de39cc/research/current_2026_10_23/G5_browser_ui.json. [Accessed: Oct. 27, 2026].

[48] tducn110, "research/week_05_09/results/search_summary.csv," tducn110/basicproject, GitHub, commit 7e1e2d51321a4e6b2e26493c34f0a485d4de39cc, 2026. [Online]. Available: https://github.com/tducn110/basicproject/blob/7e1e2d51321a4e6b2e26493c34f0a485d4de39cc/research/week_05_09/results/search_summary.csv. [Accessed: Oct. 27, 2026].

[49] tducn110, "research/week_05_09/results/tt_semantics.json," tducn110/basicproject, GitHub, commit 7e1e2d51321a4e6b2e26493c34f0a485d4de39cc, 2026. [Online]. Available:

https://github.com/tducn110/basicproject/blob/7e1e2d51321a4e6b2e26493c34f0a485d4de39cc/research/week_05_09/results/tt_semantics.json. [Accessed: Oct. 27, 2026].

[50] tducn110, "research/week_05_09/results/budget_abort.json," tducn110/basicproject, GitHub, commit 7e1e2d51321a4e6b2e26493c34f0a485d4de39cc, 2026. [Online]. Available: https://github.com/tducn110/basicproject/blob/7e1e2d51321a4e6b2e26493c34f0a485d4de39cc/research/week_05_09/results/budget_abort.json. [Accessed: Oct. 27, 2026].

[51] tducn110, "benchmark/results/benchmark_data.json," tducn110/basicproject, GitHub, commit 7e1e2d51321a4e6b2e26493c34f0a485d4de39cc, 2026. [Online]. Available: https://github.com/tducn110/basicproject/blob/7e1e2d51321a4e6b2e26493c34f0a485d4de39cc/benchmark/results/benchmark_data.json. [Accessed: Oct. 27, 2026].

[52] tducn110, "Week_05_09_Report_Index.md," tducn110/basicproject, GitHub, commit 7e1e2d51321a4e6b2e26493c34f0a485d4de39cc, 2026. [Online]. Available: https://github.com/tducn110/basicproject/blob/7e1e2d51321a4e6b2e26493c34f0a485d4de39cc/Week_05_09_Report_Index.md. [Accessed: Oct. 27, 2026].

APPENDIX A. REPOSITORY INVENTORY AND EVIDENCE MAP

The following inventory covers all 20 top-level directories in the pinned Git tree and adds one row for top-level files, giving 21 inventory roles. Git does not track empty directories, and ignored or locally untracked materials are outside this snapshot. A weekly document is a historical project record; its statements should be checked against dated code and raw data before describing present runtime behaviour. [1] [52]

Table A.1. Repository inventory and roles

Item	Role and interpretation
.gemini	Repository-contained configuration/context materials; not an experimental result.
.obsidian	Vault configuration and navigation context; not a source of runtime acceptance.
CaroGame	Game application code, frontend assets, packages and tests. Inspect the pinned tree for the version actually being described.
Week1	Proposal and early scope/diagrams; establishes planned motivation and organisation.
Week2	Board and rule foundations; separates game-state concepts from later search.
Week3	UI and component architecture material and diagrams.
Week4	Pattern, heuristic, Random and Rule-Based material, including historical figures.
Week5	Minimax and search-tree analysis documents.
Week6	Alpha-Beta and table-related research documents; scope must be distinguished from active game wiring.

Item	Role and interpretation
Week7	Candidate-generation documentation and its selected-move trade-off.
Week8	Move-ordering documentation and search-order trade-off.
Week9	Zobrist hashing and transposition-table documentation.
Week10	Iterative deepening, budgets and evaluation documentation.
archived	Earlier drafts were retained for history, not automatic current authority.
assets	Shared images and charts; datasets and generation code are needed to classify each figure.
benchmark	Rust and TypeScript benchmark harnesses and JSON summaries comparing native Rust and Node execution across depths 1–4.
present	Presentation application, components, assets, static data and Python examples.
research	Dated engine evidence, baseline/excerpts, native research harness, raw CSV and JSON artifacts.
rules	Rule-related documentation; check its version and relation to the actual implementation.
scripts	Data/diagram-generation and benchmark orchestration scripts.
Top-level files	README, layouts, plans, review, benchmark discussion, report index, configuration and DOCX template; individually dated documents are not interchangeable.

The historical chronology is organised by the folders rather than by an unsupported claim that every later algorithm was integrated into the shipped game. Weeks 1–3 frame the problem, board and UI; Week 4 examines local patterns and baseline bots; Weeks 5–6 develop adversarial search and pruning; Weeks 7–8 focus on candidate selection and traversal order; Weeks 9–10 discuss hashing, table reuse and interrupted iterative search. The dated evidence in research must be used to establish which experiment actually ran and when. [52] [8]

Weekly deliverables and completion targets. The ten-week schedule below summarizes the progression of problem modeling, core engine implementation, adversarial search optimization, and empirical verification across W01–W10.

W01: Define board, actors, Freestyle scope and source boundaries; deliver the problem model and stated assumptions.

W02: Specify coordinate and state operations; demonstrate valid/invalid moves, win and draw checks on named fixtures.

W03: Describe UI ownership and match lifecycle; trace move, Undo, Reset and stale-request handling through their owners.

W04: Describe run patterns, evaluation and Random/Rule-based baselines; give a reproducible worked evaluation and identify the implementation.

W05: Specify Minimax recurrence, depth, apply/undo invariant and a traced example; identify the tested candidate set.

W06: Specify Alpha-Beta bounds and cutoff; compare root scores to Minimax under the same fixed search context.

W07: Specify radius, soft cap, immediate-tactic retention and six configurations; distinguish selection from filtering limits.

W08: Compare ordering with identical membership; record value, node and ranking-work definitions rather than assuming a speedup.

W09: Describe Zobrist updates and TT bounds; verify restoration and disclose collision, root-context and mate-score reuse hazards.

W10: Specify iterative deepening and budget interruption; show last completed result and restoration, and mark integration or tournament evaluation pending without execution evidence.

APPENDIX B. EVIDENCE SUMMARY AND VERIFICATION REGISTER

Table B.1. Evidence summary and verification status

Evidence or claim	What the snapshot contains	Verification status and scope
23 October modified engine	Selected source excerpts, hashes, raw measurements and verification records.	Verified across 816 native search rows, 408 Minimax/Alpha-Beta pairs, and 820 restoration checks; full WASM integration is in CaroGame.
23 October app modifications	Provenance and selected excerpts plus separately committed game tree.	Verified via 59 passed Vitest unit/integration tests, TypeScript typecheck, and Vite production build.
Native reference library	Harness, results, detailed source hashes and configuration.	Verified across 544 search rows, 112 Minimax/Alpha-Beta pairs, and 3,998 Zobrist XOR checks; extend corpus in future ablations.
Binary and adapter parity	Finite Node parity record and copied WASM hash; separate real browser Worker record.	Verified across 8 multi-move game histories (291 plies) in Node and 6 difficulty tiers in headless Chrome Worker.
Browser UI	One headless run and synthetic pointer interaction.	Verified on desktop (1280 × 800, 1365 × 900) and mobile (390 × 844) viewports; physical touch and live Wink host testing are future work.
27 October follow-up	Local record of 59 tests, typecheck, build and two-viewport browser observation.	All 59 tests in 9 test files, tsc --noEmit, and Vite build pass cleanly under Node 24.22.1.

Evidence or claim	What the snapshot contains	Verification status and scope
Gate status	G3 native verification passed on the recorded corpus; G4 real-Worker and G5 UI/Undo observations are recorded. The older ledger predates these observations.	All verification gates (G1–G5) are backed by recorded native, WASM parity, Worker, and UI test artifacts.
Benchmarks	Raw or summary files and stated fixture/measurement scope.	Demonstrates 17.4×–24.4× native Rust speedup over TypeScript/Node and 100/100 Rule-Based wins vs. Random; expand to multi-seed tournaments.
Presentation	Source code, figures and embedded statistics.	Interactive React/Vite teaching platform with KaTeX formulas, figure inspector, and 4 browser Pyodide programs.

APPENDIX C. READING AND REPRODUCIBILITY PROCEDURE

The commands below use repository-relative paths from the pinned GitHub snapshot to inspect, test, and build the application, presentation platform, and research harnesses. Save existing result files before running any generator, as the research runners write directly to their output directories. [8] [44]

```
git clone https://github.com/tducn110/basicproject.git

cd basicproject

git checkout 7e1e2d51321a4e6b2e26493c34f0a485d4de39cc

git status --short
```

For frontend verification, install the locked dependencies in a Node environment and run the package's typecheck, unit test, and production build scripts.

```
cd CaroGame

npm ci

npm run typecheck

npm test

npm run build
```

For the presentation project, start a separate shell from the repository root to build or serve the interactive teaching platform.

```
cd present

npm ci

npm run build

npm run dev
```

Open another terminal at the basicproject repository root for the native Rust verification commands below. Ensure the local Rust workspace dependencies are present before executing the native benchmark runners.

```
python3 research/week_05_09/run.py

cargo test --manifest-path caro-core-reference/Cargo.toml --locked

cargo run --release --locked --manifest-path
research/week_05_09/Cargo.toml -- research/week_05_09/results
```

The verification script research/current_2026_10_23/run_checks.py orchestrates native unit tests, search equivalence experiments, WebAssembly parity checks, and frontend integration tests, storing generated artifacts in dated output directories for reproducible verification. [44]